
Internet Boom and Emergence of NoSQL

By
Dr. Parteek Kumar

Objectives of Big Data and NoSQL

- To discuss emergence of huge data due to Internet Boom
 - To discuss major issues of Relational Databases
 - To understand why NoSQL is required
-

Objectives of Big Data and NoSQL

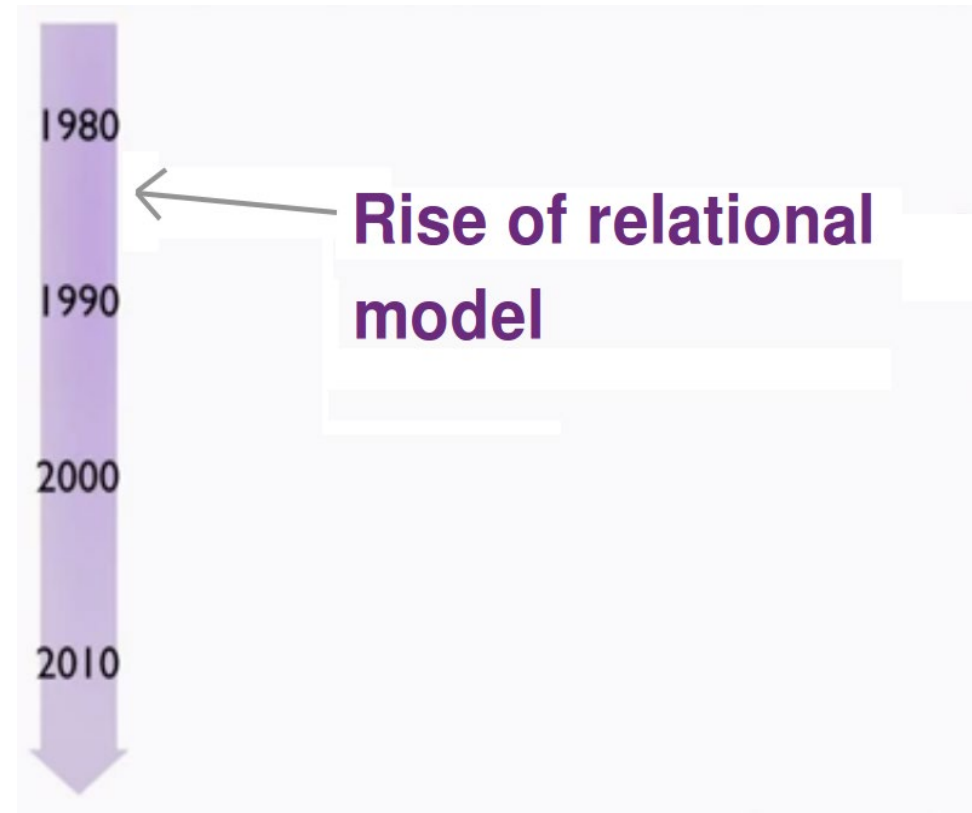
- To discuss major issues of Relational Databases
- To understand why NoSQL is required
- To comprehend NoSQL characteristics
- To understand different Data Models of NoSQL
- To understand the concept of the CAP Theorem
- To discuss the future of NoSQL

The rise of Relational Databases

- In the year 1969, Dr. E.F Codd proposed the relational model.
- It was soon adopted by the mainstream software industries due to its simplicity and efficiency replacing hierarchical and network models that were prevalent at that time.
- The timeline showing the rise of the relational model is depicted in figure.

The rise of Relational Databases

- Rise of the relational model
- The reasons for the success of relational databases were its simplicity, power of SQL, support for transaction management, concurrency control, and recovery management.



Major issues of Relational Databases

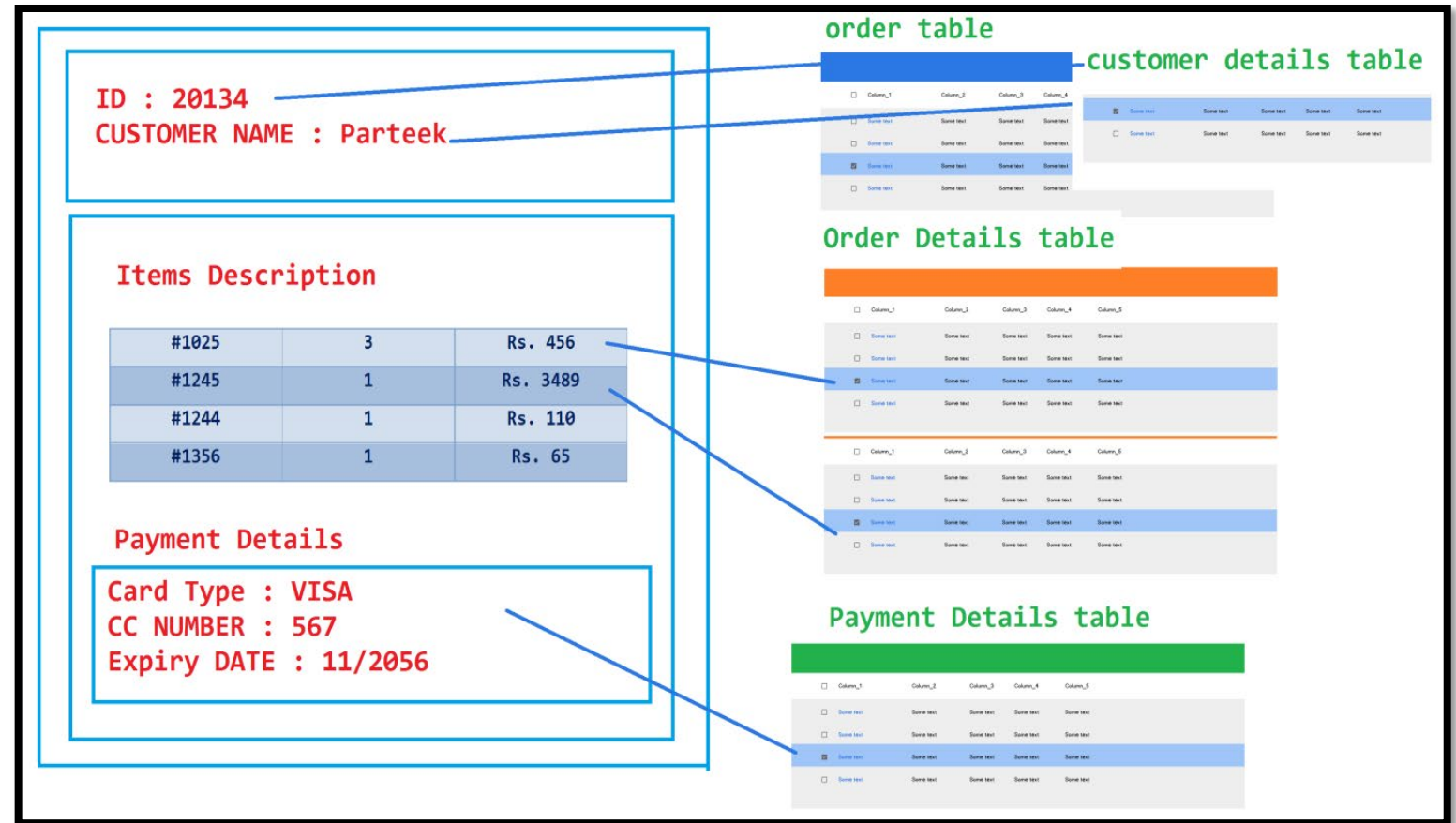
- The relational data model organizes data in rows and columns arranged in a tabular form.
- In the relational model, a row is known as a tuple which is a collection of key-value pairs and the collection of tuples is known as relation.
- Every single SQL operation operates upon relation and hence the output of each query is also a relation.
- This classic rule though sounds simple but has limitations to it, like SQL assumes each of the entry in a relational tuple to be atomic in nature i.e. simple, which can't be divided further.
- Hence, each of these tuple entries can't work that elegantly if it contains a list of items or a nested record.

Major issues of Relational Databases

- Whereas, in-memory data structures don't have this limitation and hence, can contain much richer structures, as their entries.
 - But use of in-memory data structure causes another problem of impedance mismatch, as it requires the in-memory data structures to be converted into relational structure, in order to store them on the disk.
 - Hence, the inter-translation of two representation causes impedance mismatch as shown in next slide.
-

Major issues of Relational Databases

- A single structure of an Order is composed of multiple tuples from different tables from some relational database



Major issues of Relational Databases

- Impedance mismatch was more than a suffering, and caused a lot of frustrations in developers who were developing applications at that time.
 - They even thought that impedance mismatch would be one of the major cause in the replacement of relational databases with other databases that are capable of replicating the in-memory data structures to disk.
 - Later, with the advent of object-oriented programming languages, object-oriented databases were also developed but they could never make their way into the mainstream software development.
-

Major issues of Relational Databases



Rise of Object Databases

Major issues of Relational Databases

- Object Oriented Languages were a huge success, and no doubt, they are still the most popular programming paradigm, but this couldn't happen in the case of Object Oriented Databases which vanished from the market.
- This led to the development of object-relational mapping (ORM) frameworks which removed a lot of grunt work. However, ORM framework brought its own limitations.
- The moment programmer's abstracted some database using ORM framework, they had to compromise with the query performance.

Major issues of Relational Databases



Relational dominance in the late 1990s and early 2000s

Major issues of Relational Databases

- Nevertheless, the Enterprise computing world witnessed relational databases dominating the market till the last decade of 20th century but, the dominance soon subsided with the beginning of 21st century.

Challenges from the Internet boom

- In late 1990s the Internet grew at an unimaginable pace. Soon large data sets appeared in the form of social networks, activity logs, scientific data etc.
- As the Internet became more accessible, the number of users grew which led to the exponential growth of data. This growth of data is depicted in next slide.

Challenges from the Internet boom



Generation of lots of traffic during the Internet boom

Challenges from the Internet boom

- **The Rapid Growth of Unstructured Data**
- **Types of Data in the era of the Internet boom**

The Rapid Growth of Unstructured Data

With the Internet, there was a rapid growth of unstructured data. The volume of unstructured data can be clearly observed from the following facts and figures.

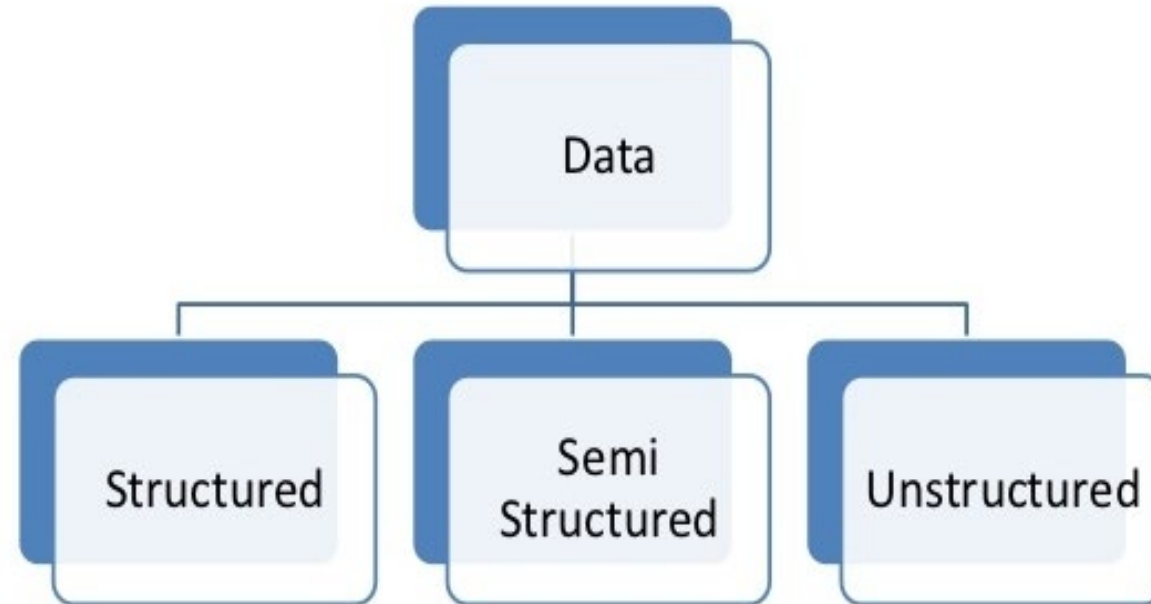
- 48 hours of YouTube videos are daily uploaded every minute.
 - Daily more than 50 websites are launched every minute.
 - Last 2 years holds the credit of 90% contribution to data.
 - 2020 will witness the rise in data production by 44 times to that of 2009.
-

The Rapid Growth of Unstructured Data

With the Internet, there was a rapid growth of unstructured data. The volume of unstructured data can be clearly observed from the following facts and figures.

- On Facebook, more than 35,000 likes are given to different brands and organizations.
 - More than 100 TB of data is uploaded daily on Facebook.
 - According to Twitter's own research, it saw roughly 185 million tweets every day and had more than 465 million accounts.
 - Monthly 30 billion pieces of content are shared through Facebook.
 - IDC Digital Universe report indicated the generation of around 1.8 zettabytes of data.
-

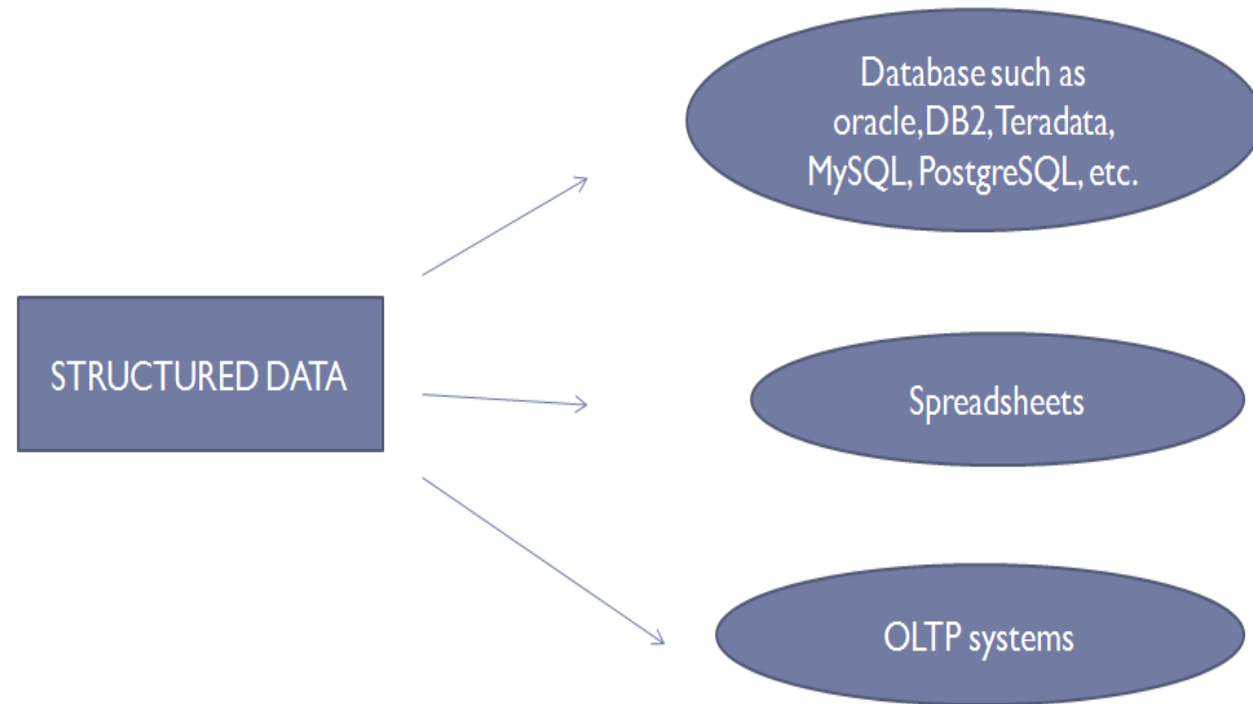
Types of Data in the era of the Internet boom



Structured Data

- The data that has a fixed structure and can be arranged in rows and columns. For example, a database stored in SQL, excel sheets, OLTP (Online Transaction Processing) data, etc are all structured.
 - Structured data has a relational key and hence mapping into pre-defined schema is quite easy. Even , the management of information is simple here. Out of all the data present in world, data which falls under the category of structured data is around 5-10%
-

Structured Data



Semi-Structured Data

- Semi-structured data can't be accommodated into a relational database but due to some organizational attributes or properties , analysis of semi-structured data is quite easier.
 - All the CSV file, XML and JSON documents fall under the category of semi-structured data.
 - Just like structured data, the semi-structured data also represent only about 5-10% of all data.
-

Unstructured Data

- Majority of data in the world is Unstructured data. Numerically around 80% of the data generated across the globe is unstructured.
 - It mostly includes text and multimedia content, for example :business docs, e-mail messages, word documents, images, audio-clip , video files, presentations and even web pages are unstructured data.
 - It is the most abundant form of data, in fact most of us conduct our lives around unstructured data.
-

Unstructured Data

- Let us see some more examples of unstructured data.
 - **Satellite images:** All the weather data or the data fetched from satellites surveillance is included under this. You can search about Google Earth to see satellite image data.
 - **Scientific Research data:** Right from atmospheric data to high energy physics data (data generated from particle accelerators). This kind of data is majorly generated in practical research domains.
 - **Videos and Photos:** Images and videos collected during traffic surveillance.
-

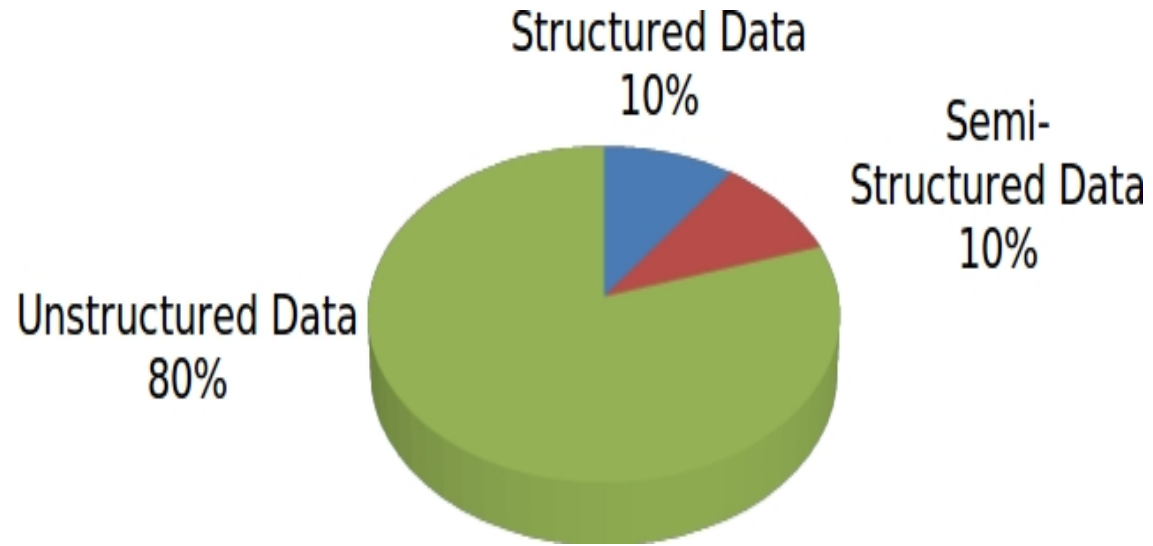
Unstructured Data

- **RADAR or SONAR data:** Data generated during the vehicular, meteorological, and oceanographic seismic profile studies.
 - **Text internal to any company:** Any text underlying in official documents, reports, survey logs, results, and email conversations. In fact, the data specific to enterprises constitutes text information in larger percentage.
-

Unstructured Data

- **Data from Social Platforms:** Right from Twitter to Flickr, most of the data generated from numerous social platforms (like LinkedIn, Facebook and others) is unstructured in nature.
 - **Mobile data:** Data such as text messages, chats, location pin-ups.
 - **Website content:** Even websites like Vimeo, YouTube, Voot generate lot of unstructured data.
-

Types of Data in the era of the Internet boom



Challenges from the Internet boom

- **Test Your Skills**
- Classify the following words into structured, unstructured or semi-structured data.

Email	Chat conversations
MS Access	Relations / Table
Images	Facebook
MS Excel	Database

Challenges from the Internet boom

The solution is given below:

Structured	Unstructured	Semi-structured
MS Access	Email	XML
Database	Images	
Relations / Table	Chat conversations	
MS Excel	Facebook	

Unstructured Data

- ❑ Since 80% of data is unstructured data so there is a huge demand for database experts who can process and analyze this data.
 - ❑ It is important to note that traditional databases can only process structured data which is merely 10% of all data.
 - ❑ Thus, the Internet boom produced new challenges and opportunities in terms of technologies and skills.
-

Big Data and NoSQL

Emergence of Big Data due to Internet Boom

- This huge volume of data gave rise to a new concept known as Big Data.
 - Gartner's definition of Big Data says, that - "Big data is high-volume, high-velocity and/or high-variety information assets that demand cost-effective, innovative forms of information processing that enable enhanced insight, decision making, and process automation."
-

Emergence of Big Data due to Internet Boom

- The big data involves 4 V's as given below.
 - Volume: Data size
 - Variety: Data formats
 - Velocity: Data streaming speeds
 - Veracity: Data trustworthiness
-

Emergence of Big Data due to Internet Boom

Volume of Data

- Big Data is the data generated in humongous amount. It is the enormous data generated through machines, network and even human conversation over different channels such as social media.

Emergence of Big Data due to Internet Boom

Volume of Data

- A glimpse of data sizes in terms of bytes has been given below.
 - ❑ 10^3 Bytes- 1 KB (Kilobytes)
 - ❑ 10^6 Bytes- 1 MB (Megabytes)
 - ❑ 10^9 Bytes- 1 GB (Gigabytes)
 - ❑ 10^{12} Bytes- 1 TB (Terabytes)
 - ❑ 10^{15} Bytes- 1 PB (Petabytes)
 - ❑ 10^{18} Bytes- 1 EB (Exabytes)
 - ❑ 10^{21} Bytes- 1 ZB (Zettabytes)

Emergence of Big Data due to Internet Boom

Variety of Data

- Variety of Data refers to data coming from different sources or data in different formats. Traditionally, most of the data was yielded from spreadsheets and databases. Whereas, in present scenario the data can come from e-mails, sensor devices, images or photos, videos, monitoring devices, PDFs, audio, etc.
 - Hence, storing or performing any kind of operation such as mining or analysis on a variety of unstructured data is cumbersome and often results into problems and errors.
-

Emergence of Big Data due to Internet Boom

Velocity of data

- Data is generated from different sources and the speed at which the data is generated could be ascribed as the Velocity of Data. Right from social networking sites to networks, data flows from multitude of sources. This flow of data is massive and continuous.
 - To understand the velocity of data we can consider the case of a driverless car. To make a driverless car we have to deploy thousands of sensors and the frequency at which these sensors are generating data is in milliseconds. We have to process this high-speed data in real time and generate the response in order to make the whole thing work.
 - This example clearly indicates the importance to process high-velocity data.
-

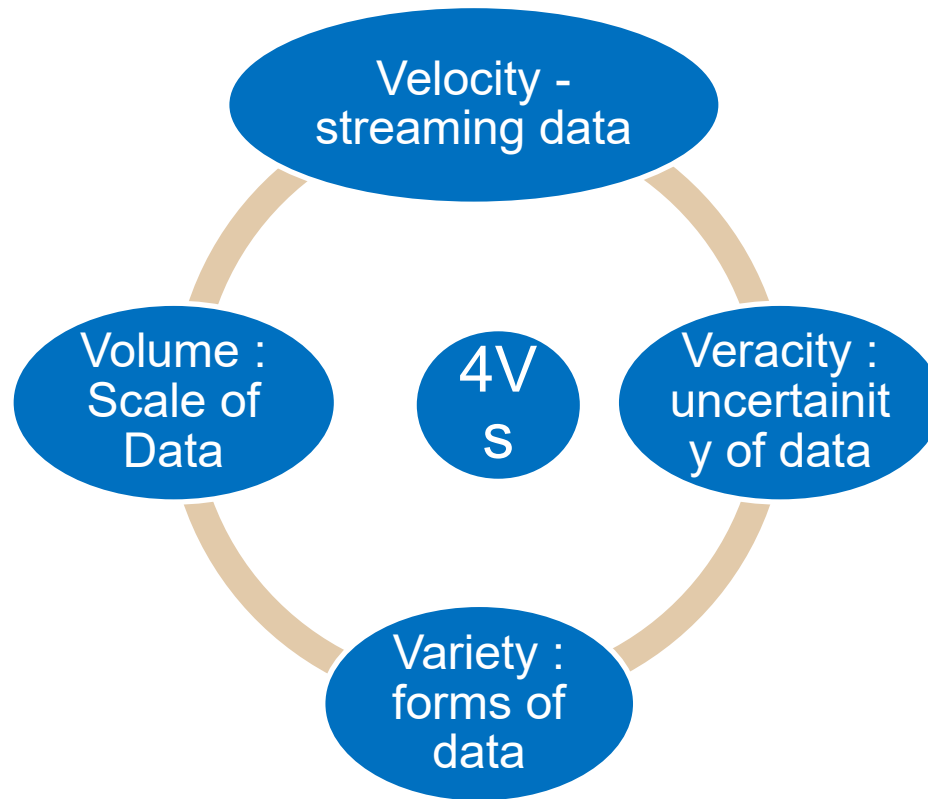
Emergence of Big Data due to Internet Boom

Veracity of data

- Noisy data and all kinds of data abnormalities are collectively referred by the term Veracity. It tells us about the trustworthiness of data.
- To judge the veracity, we ask a simple question - "Is the data that is being stored, and mined meaningful to the problem being analyzed?" It is estimated that \$3.1 Trillion a year is wasted in the US economy due to poor data quality.
- Compared to volume and velocity, the veracity of data analysis is supposed to be the biggest challenge.

Emergence of Big Data due to Internet Boom

These 4 V's of big data are very well illustrated by the info graphics.



Possible solutions to handle huge data emerged from the Internet boom

- Whenever new challenge arises, we first try to solve it via existing technologies and skills.
 - During the Internet boom relational database was at dominant. Hence, the Big Data challenges or problems were solved using the relational database.
 - The first solution was to scale up resources. Scaling up refers to the process of including bigger machines, more number of processors, increasing disk storage and memory,
-

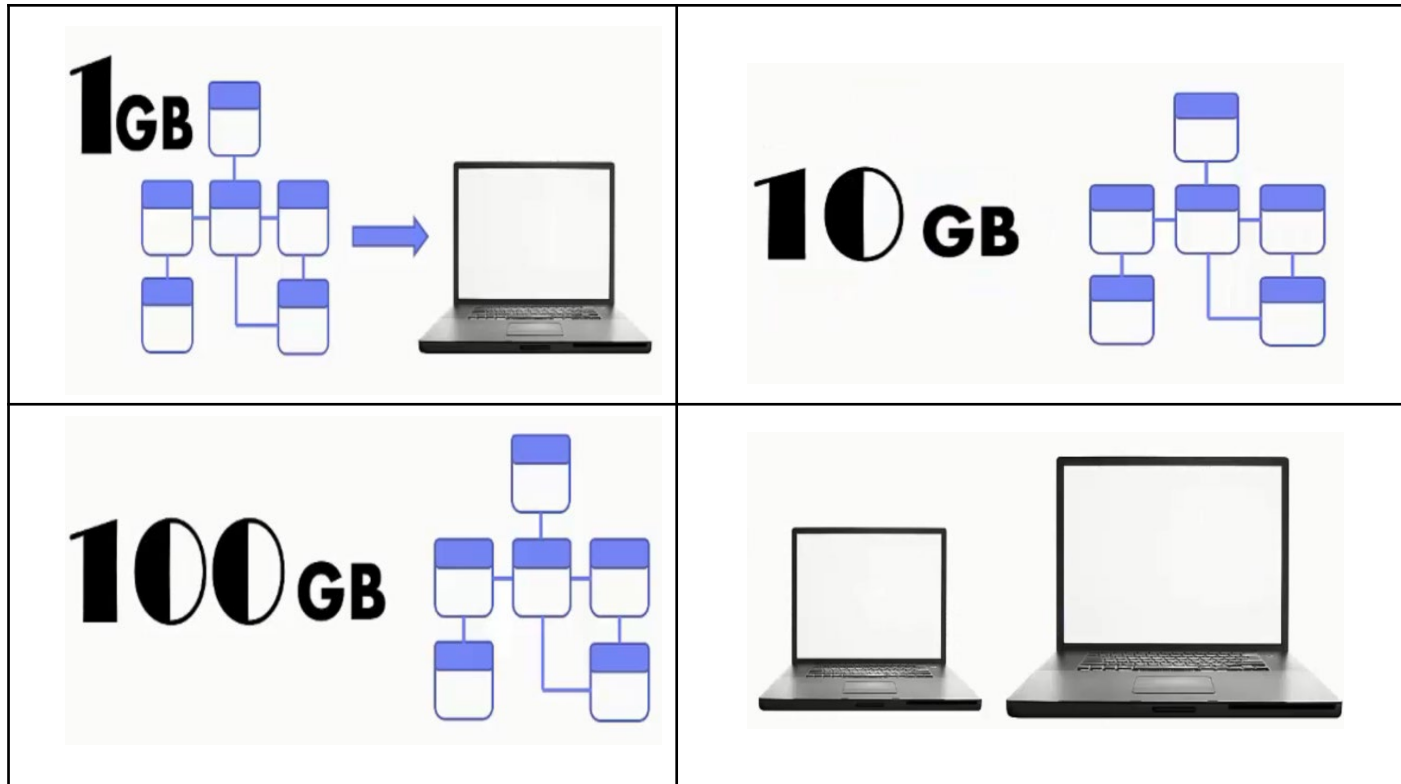
Possible solutions to handle huge data emerged from the Internet boom

- Scaling Up using large centralized server.
- But this lead to increase in price as adding more computing resources definitely cost some extra bucks and above all there is always an acme till which you can increase size or add resource, after a point everything has a saturation



Possible solutions to handle huge data emerged from the Internet boom

- Handling of huge data through the relational model



Possible solutions to handle huge data emerged from the Internet boom

- But soon we reached our limits. The alternative solution was to scale out i.e. the process of using numerous small machines in a cluster. Hence, a cluster can use commodity hardware and this doesn't add much to the cost as it happens in the case of large servers.
 - Encountering failures in individual machines is quite common, but the overall cluster can be built to keep going despite such failures, providing very high reliability and hence these are resilient in nature.
-

Possible solutions to handle huge data emerged from the Internet boom



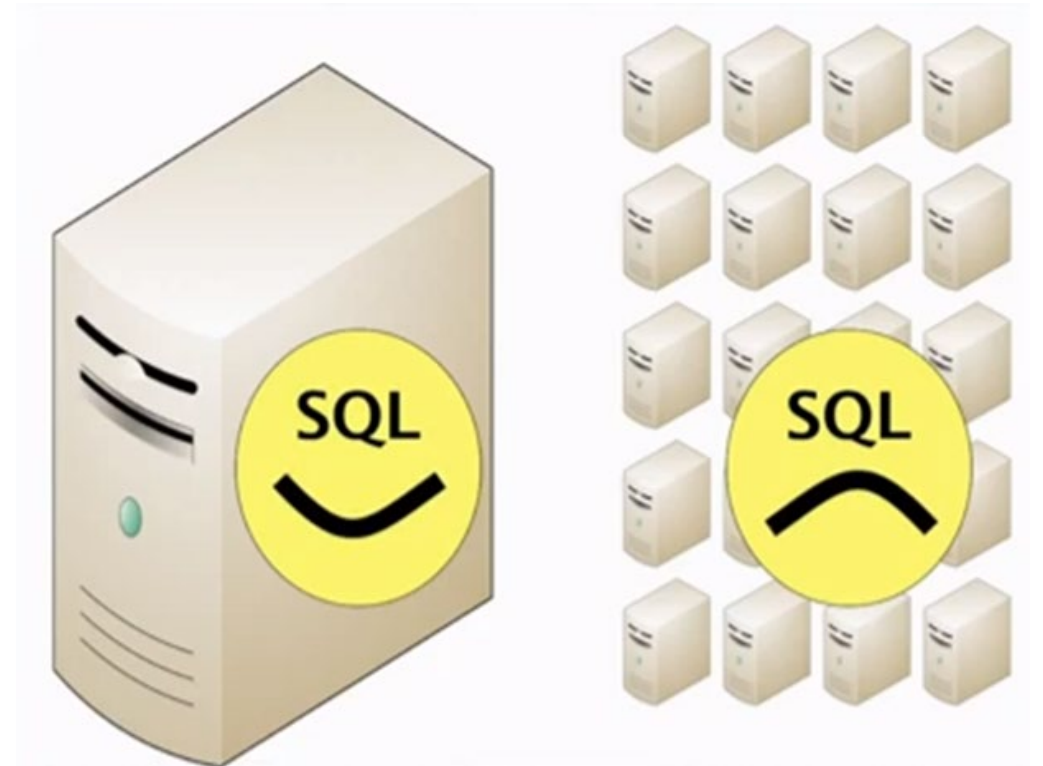
- Cluster Computing emerged as a winner.

Possible solutions to handle huge data emerged from the Internet boom

- More number of organizations switched to clusters, and this transition revealed a new problem of relational databases, that is they were not designed to run efficiently on clusters.
 - Some of the popular clustered relational databases, such as Oracle RAC or Microsoft SQL Server, use the concept of a shared disk subsystem and if we tune relational database for cluster environment, it is considered as “unnatural acts.”
 - This was accompanied by **some licensing issues too. Whereas, the commercial relational databases were usually priced on a single-server basis, and hence running them on a clusters, led to an increase in price** and this further caused frustrating negotiations with purchase departments.
-

Possible solutions to handle huge data emerged from the Internet boom

- Thus the need of an alternative route to data storage was felt within the organization.



SQL in Cluster Environment

The emergence of Technologies for Cluster Environment

- Google and Amazon started their work based on cluster environment and had already implemented various cluster technologies.
 - They had already implemented and were running large clusters. Furthermore, they were capturing huge amounts of data. All this forced them to develop an alternative solution to relational databases.
 - As the 2000s drew on, both Google and Amazon presented a brief of research in form of research papers titled as BigTable from Google and Dynamo from Amazon
-

The emergence of Technologies for Cluster Environment



BigTable and Dynamo for Cluster Environment

Birth of NoSQL

- The usage of the term “NoSQL” that is known to us today can be traced back to a developers meet organized on June 11, 2009, in San Francisco by Johan Oskarsson, a software developer based in London.
 - Big Table and Dynamo had already become a huge source of inspiration for developers around the world, this led to discussion on alternative data storage to be mandatory in every good software conference.
-

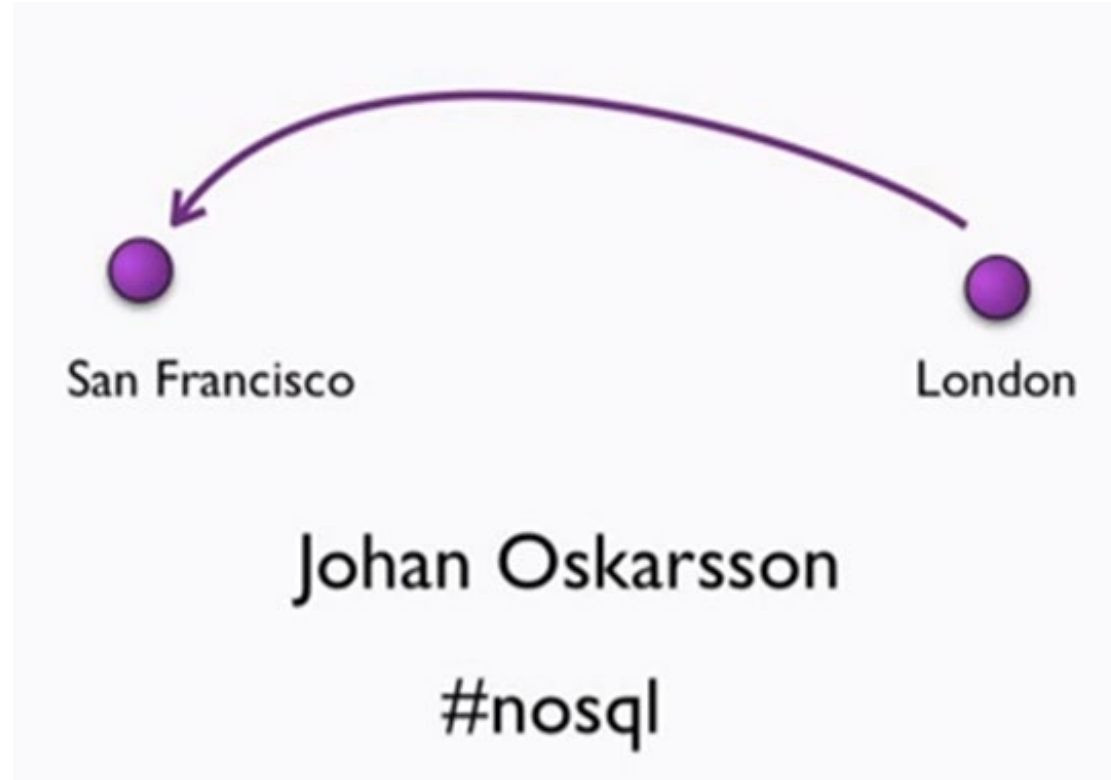
Birth of NoSQL

- Johan who was attending Hadoop summit in San Francisco, was interested in finding out more about some of these new databases, and thus he wanted to attend each of these conferences but due to less time in hand, he couldn't do so and hence he decided to host a meet where developers across the globe will present their work.

Birth of NoSQL

- In order to bring out some good and catchy name of the meet, which would make a trending hash tag on Twitter and is crisp and short with lesser number of Google hits, he took suggestions from **#cassandra** IRC channel **and after considering few names he chose "NoSQL" which was suggested by Eric Evans.**
 - Though the name was sounding negative and wasn't suffice enough to describe system properties, yet it was chosen, **as it stood firmly on the grounds of hash tag consideration.** Nobody expected a single meet's name to be one of the most popular technology jargon today.
-

Birth of NoSQL

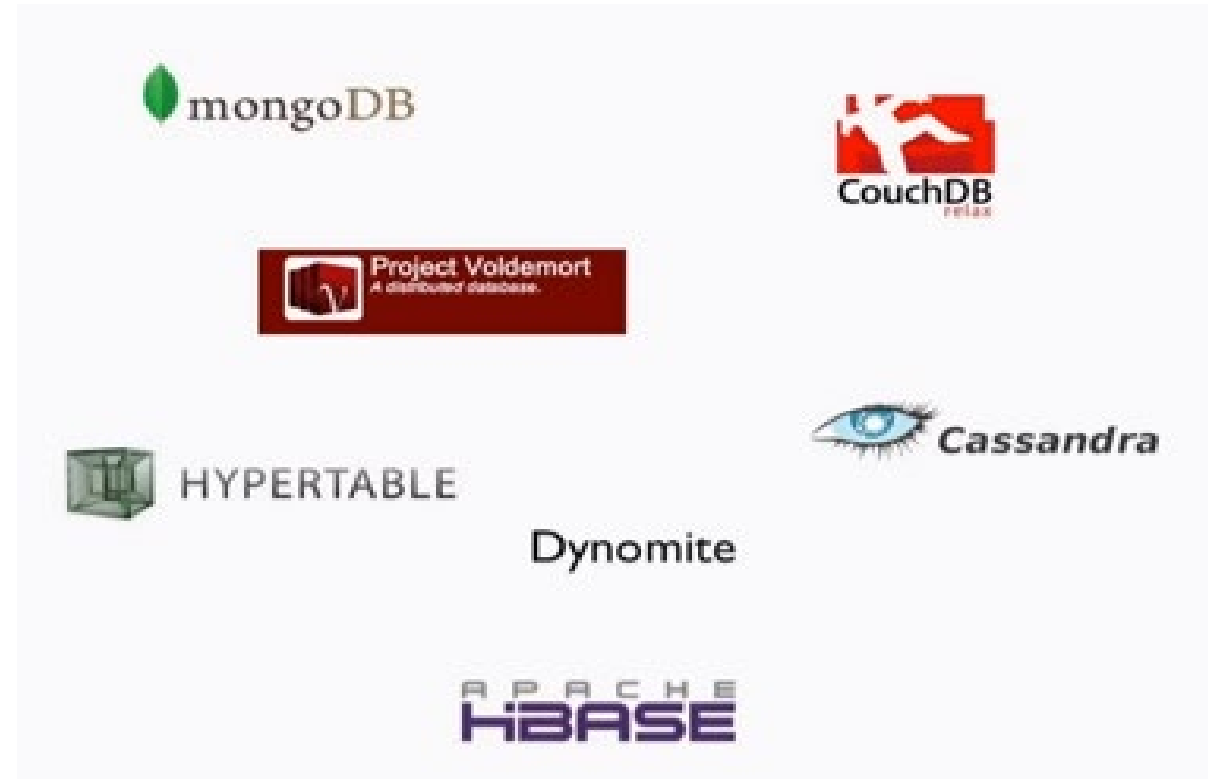


NoSQL Meet

Birth of NoSQL

- The popularity of the term “NoSQL” started skyrocketing, but it never had a strong technical definition. As the original call for the meetup asked for “open-source, distributed, non-relational databases” hence NoSQL is attached to all these features.
 - In the #nosql meet, experts from CouchDB, Cassandra, Dymomite, Hypertable, HBase, MongoDB and Voldemort participated.
-

Birth of NoSQL



Participants of NoSQL meet

Defining NoSQL from the characteristics it shares

There is no generally accepted or standardized definition of the term NoSQL. Based upon the characteristic study of any database, it is tend to be called as “NoSQL.”

The important characteristics of NoSQL are given below.

- It is not based on the relational model, thus, it is non-relational.
 - These databases are generally open-source.
 - Most of the NoSQL databases are intended to run on servers. Thus, they are cluster-friendly.
 - NoSQL databases are used to address the needs of the early 21st-century web.
 - NoSQL databases don't have a schema, hence they allow you to freely add fields to existing database records. In the case of non-uniform data this is a very crucial feature.
-

Defining NoSQL from the characteristics it shares

Characteristics of NoSQL

non-
relational open-source
cluster-friendly
21st Century
Web
schema-less

Features of NoSQL

Some Misconceptions about NoSQL

- A general misconception about NoSQL is that it does not use SQL. This is not true. Most of NoSQL databases have their own query language. For example, Cassandra uses CQL which is exactly like SQL.
 - Another important point to note is that “NoSQL,” does not mean “no” to SQL? **Rather NoSQL is a smart acronym for “Not Only SQL.”** Hence, rather than writing "NoSQL" , one should write "NOSQL".
-

What is NoSQL?

- The NoSQL name was introduced during an open-source event on distributed databases
- NoSQL doesn't mean 'No SQL'
NoSQL means 'Not only SQL'

NoSQL



'Not only
SQL'

Data Models of NoSQL

- A data model allows us to perceive and manipulate our data. It tells us about the way a client interacts with the data in a Database. Relational data model had a great dominance in last few years, it could be visualized through collection of tables/relations or like spreadsheet pages.
- Transition to NoSQL assures a definite shift from relational data model. With NoSQL other data models appeared; mainly categorized as follows:
 - Key-value data model
 - Document data model
 - Column-family model
 - Graph data model

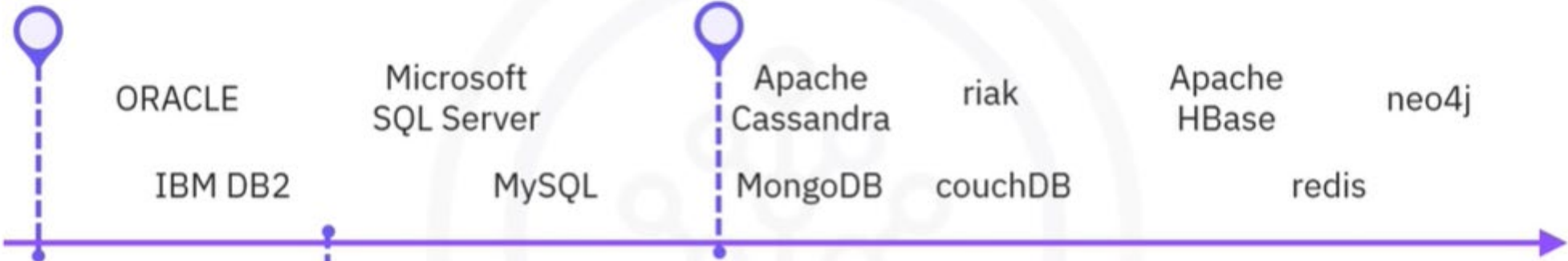
Data Models of NoSQL

- These models and systems based on these models are



History of NoSQL

1970–2000: Mainly RDBMS solutions



2005–2010: New open source and mainstream databases

2000–2005: The dotcom boom; the start of new-scale solutions; the start of NoSQL development

2010: Adoption of cloud->DBaaS

Why NoSQL?

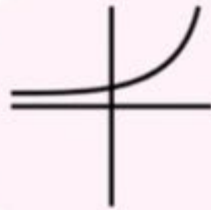


Flexible data
models

Why NoSQL?



Flexible data
models

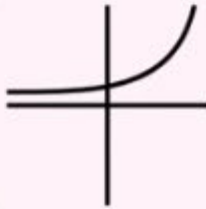


Built-in
scalability

Why NoSQL?



Flexible data
models



Built-in
scalability

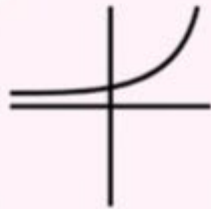


Developer
productivity

Why NoSQL?



Flexible data
models



Built-in
scalability



Developer
productivity



Distributed
databases

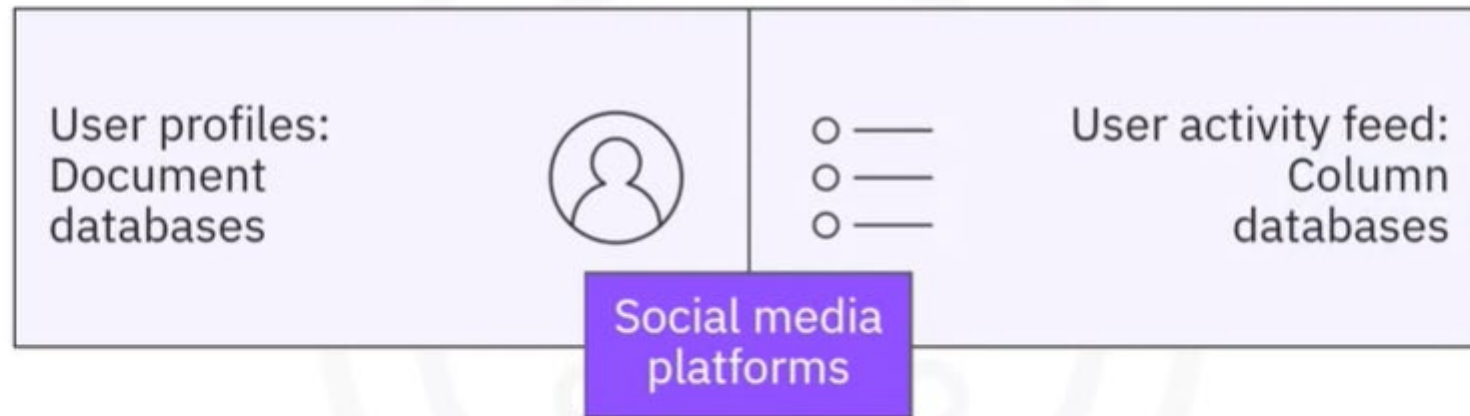
NoSQL in use: Social media example

Businesses can use a mix of NoSQL databases



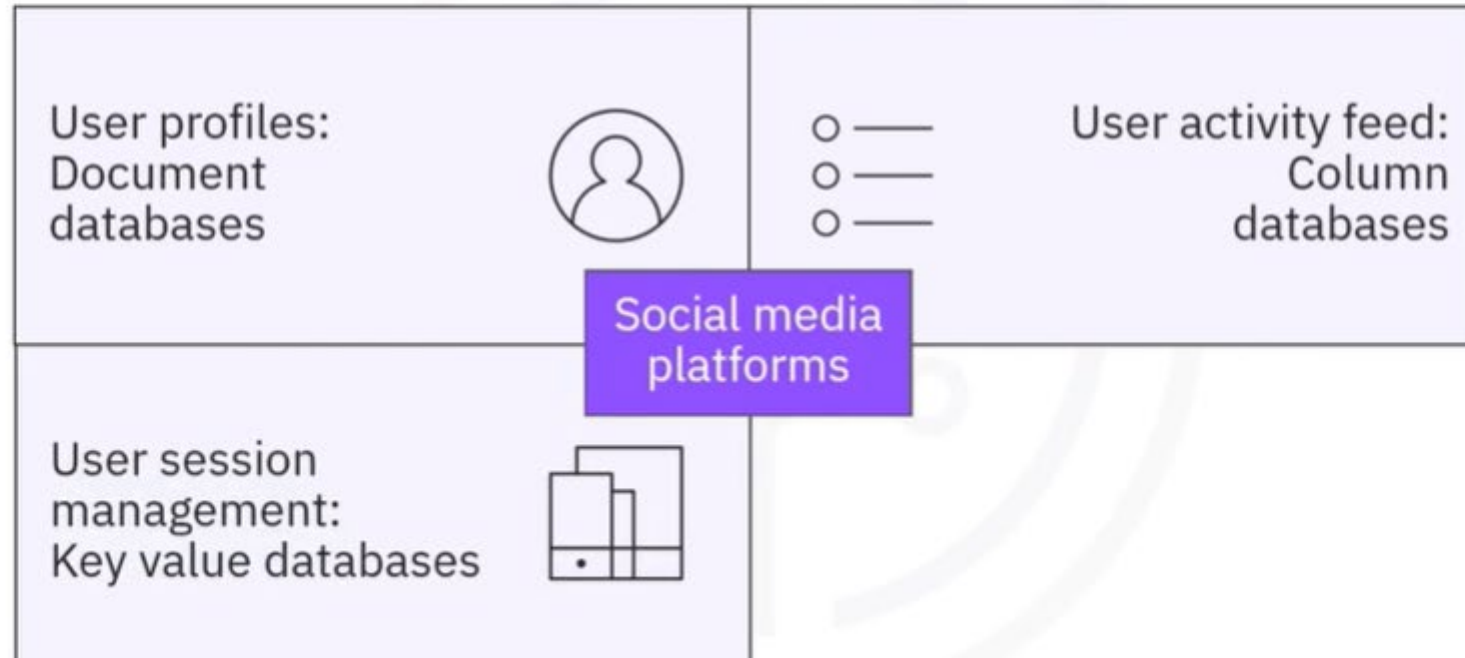
NoSQL in use: Social media example

Businesses can use a mix of NoSQL databases



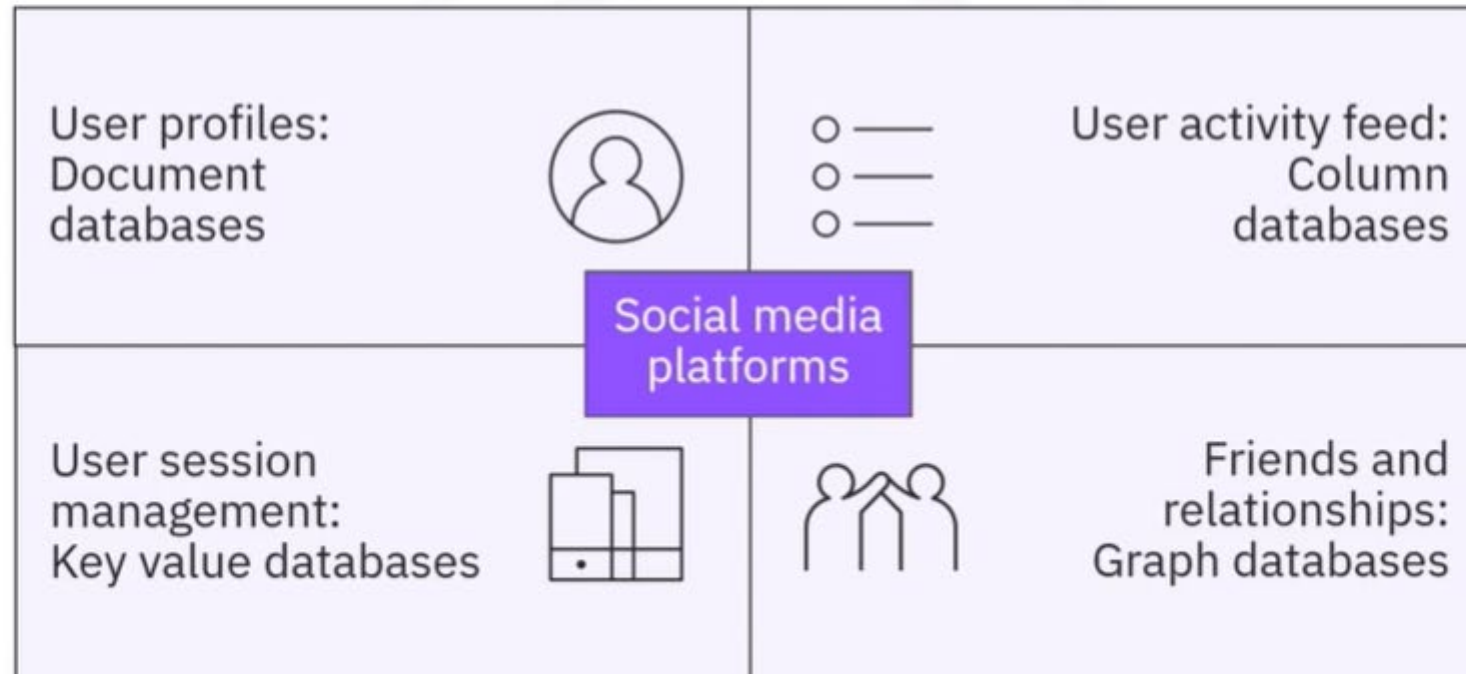
NoSQL in use: Social media example

Businesses can use a mix of NoSQL databases



NoSQL in use: Social media example

Businesses can use a mix of NoSQL databases



NoSQL database characteristics

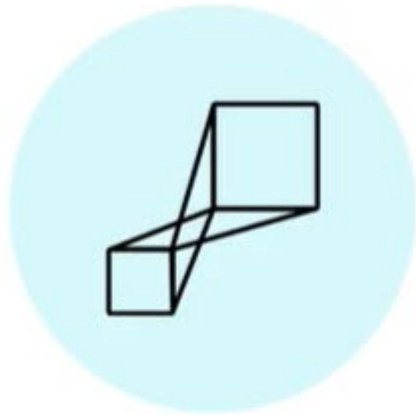
- Differ technically, but have a few commonalities
- Most NoSQL databases:
 - Are built to scale horizontally
 - Share data more easily than RDBMS
 - Use a global unique key to simplify data sharding
 - Are more use case specific than RDBMS
 - Are more developer friendly than RDBMS

NoSQL database characteristics

- Differ technically, but have a few commonalities
- Most NoSQL databases:
 - Are built to scale horizontally
 - Share data more easily than RDBMS
 - Use a global unique key to simplify data sharding
 - Are more use case specific than RDBMS
 - Are more developer friendly than RDBMS
 - Allow more agile development via flexible schemas

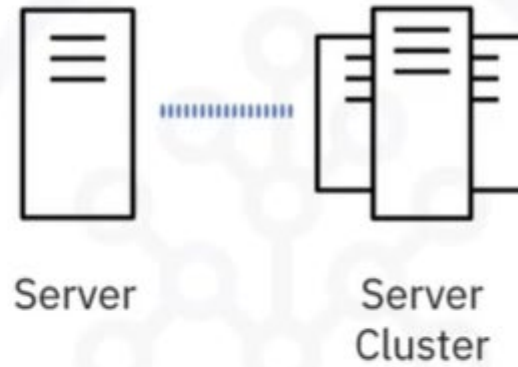
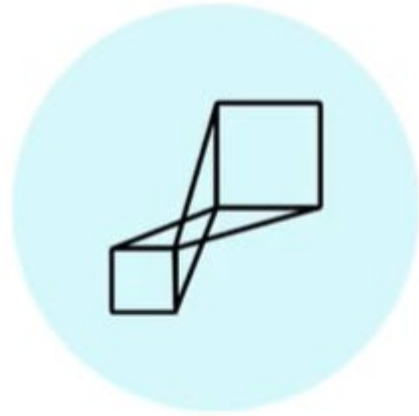
Benefits of NoSQL databases

Scalability



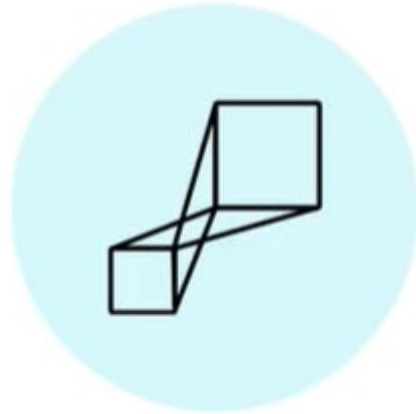
Benefits of NoSQL databases

Scalability

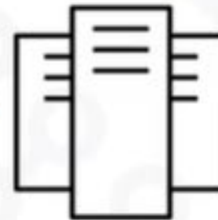


Benefits of NoSQL databases

Scalability



Server



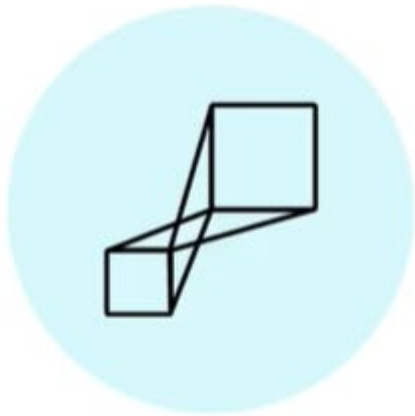
Server
Cluster



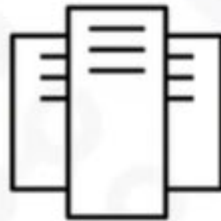
Server
Racks

Benefits of NoSQL databases

Scalability



Server



Server
Cluster



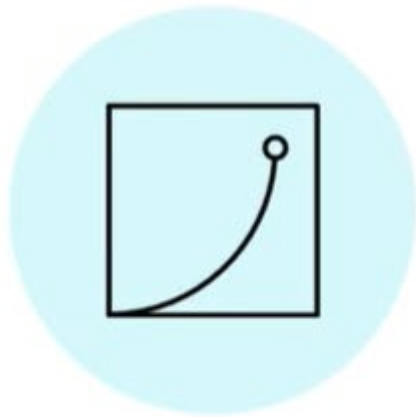
Server
Racks



Data
Centers

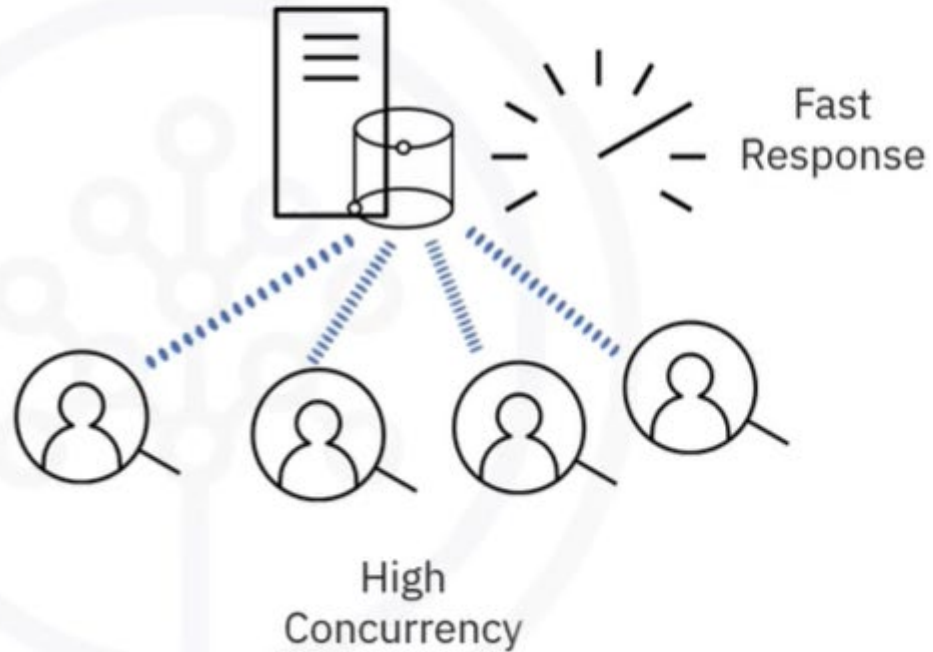
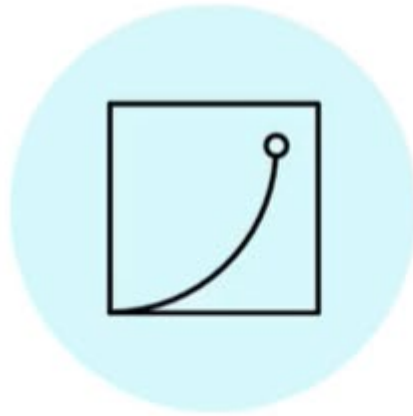
Benefits of NoSQL databases

Performance



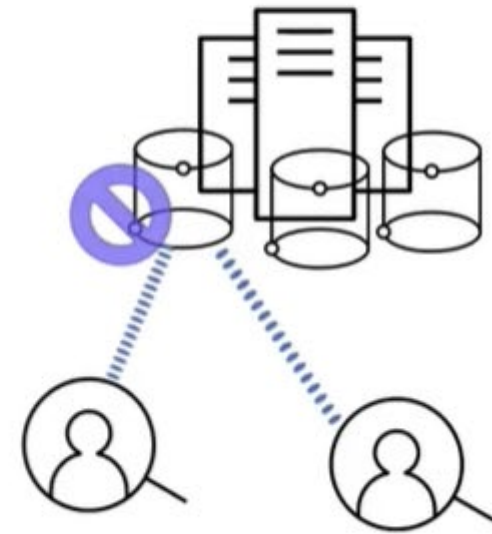
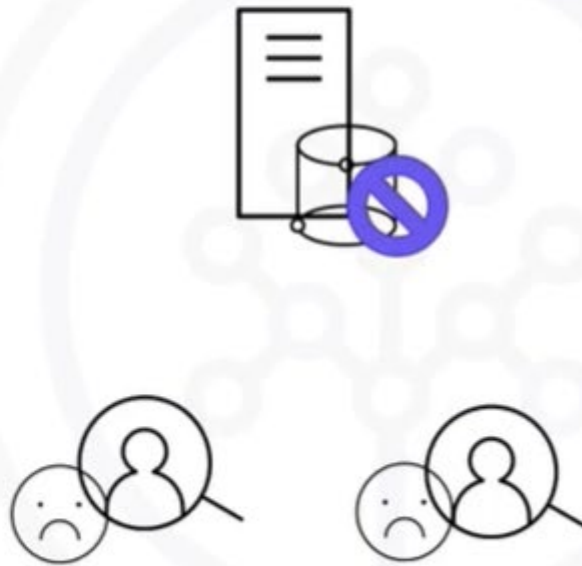
Benefits of NoSQL databases

Performance



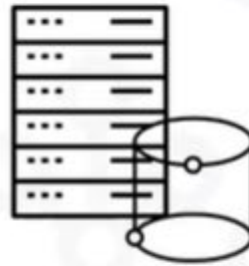
Benefits of NoSQL databases

Availability



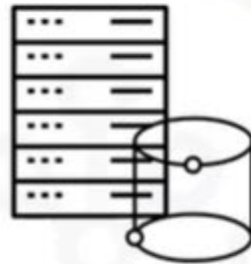
Benefits of NoSQL databases

Cloud Architecture



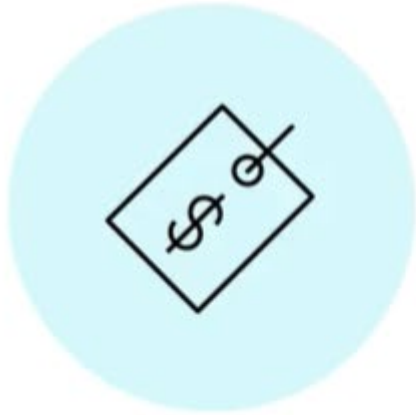
Benefits of NoSQL databases

Cloud Architecture



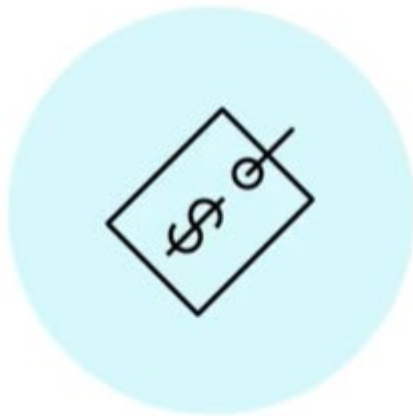
Benefits of NoSQL databases

Cost



Benefits of NoSQL databases

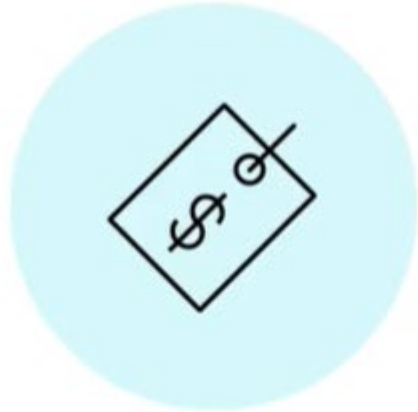
Cost



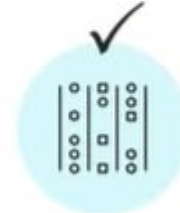
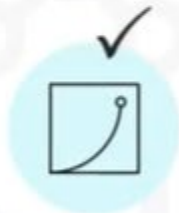
 NoSQL

Benefits of NoSQL databases

Cost

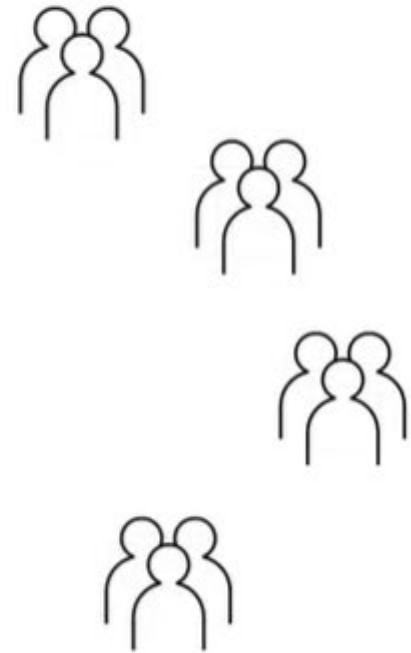


 NoSQL



Benefits of NoSQL databases

Flexible Schema



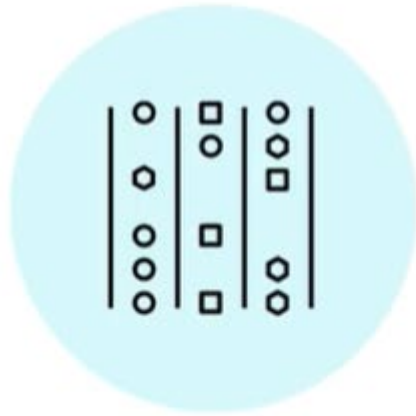
Benefits of NoSQL databases

Flexible Schema



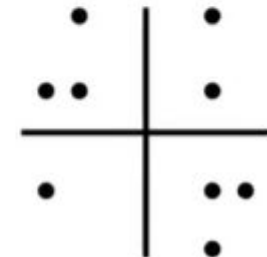
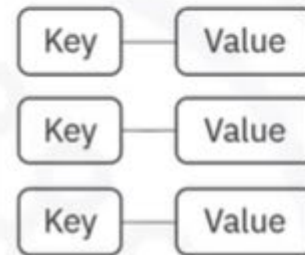
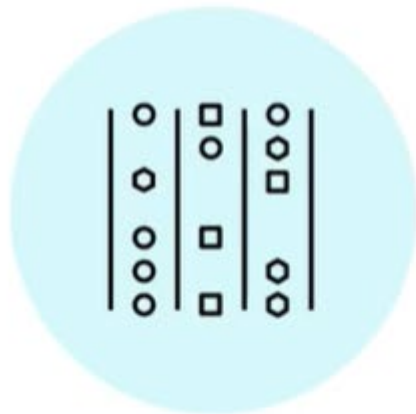
Benefits of NoSQL databases

Varied Data Structures



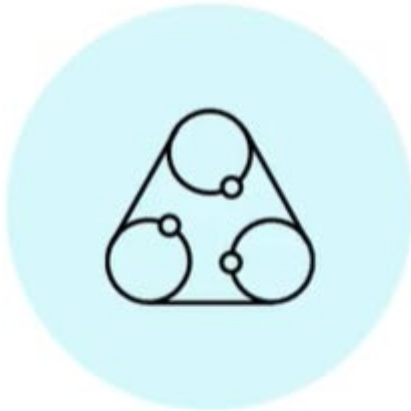
Benefits of NoSQL databases

Varied Data Structures



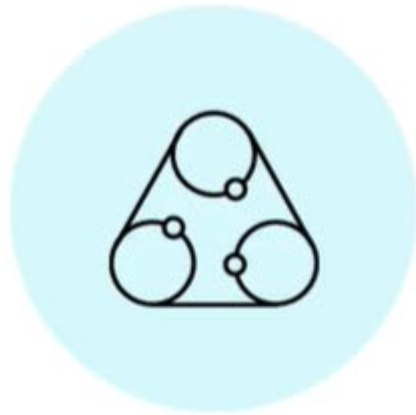
Benefits of NoSQL Databases

Specialized capabilities

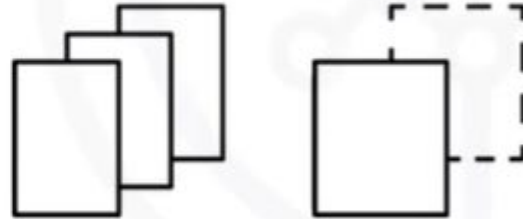


Benefits of NoSQL Databases

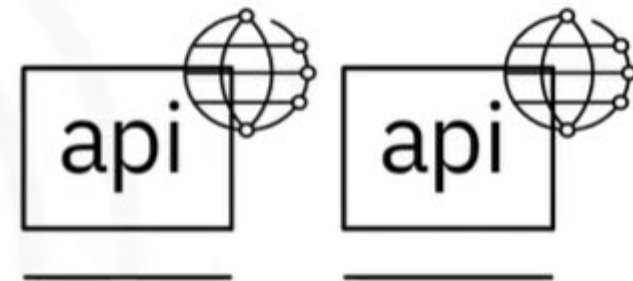
Specialized capabilities



Indexing and Querying



Data Replication Robustness



Modern HTTP APIs

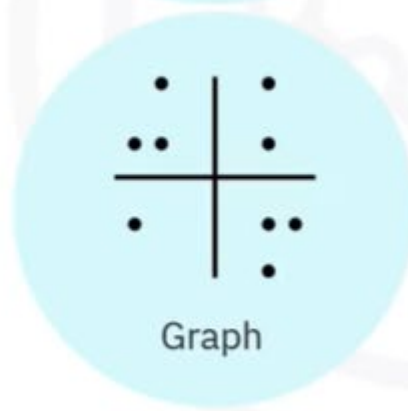
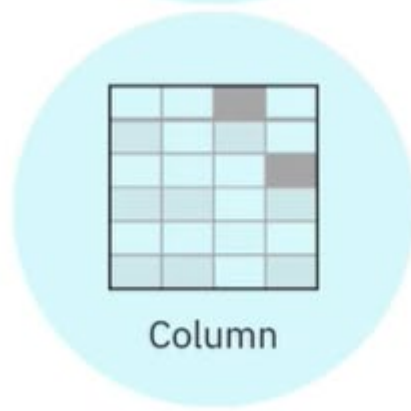
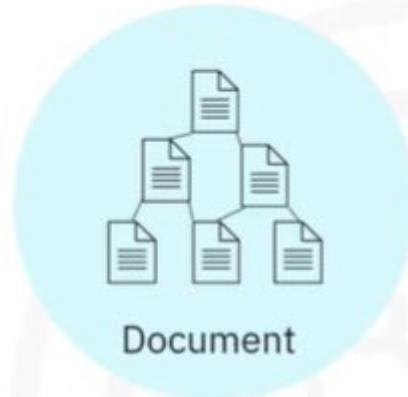
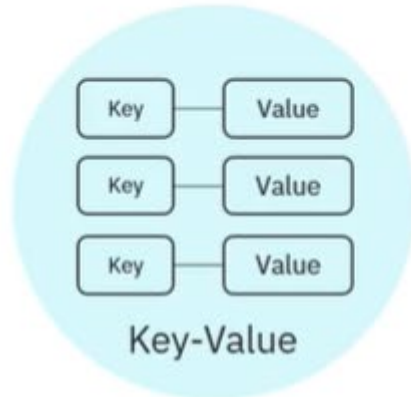




Skills
Network

Key-Value NoSQL
Databases

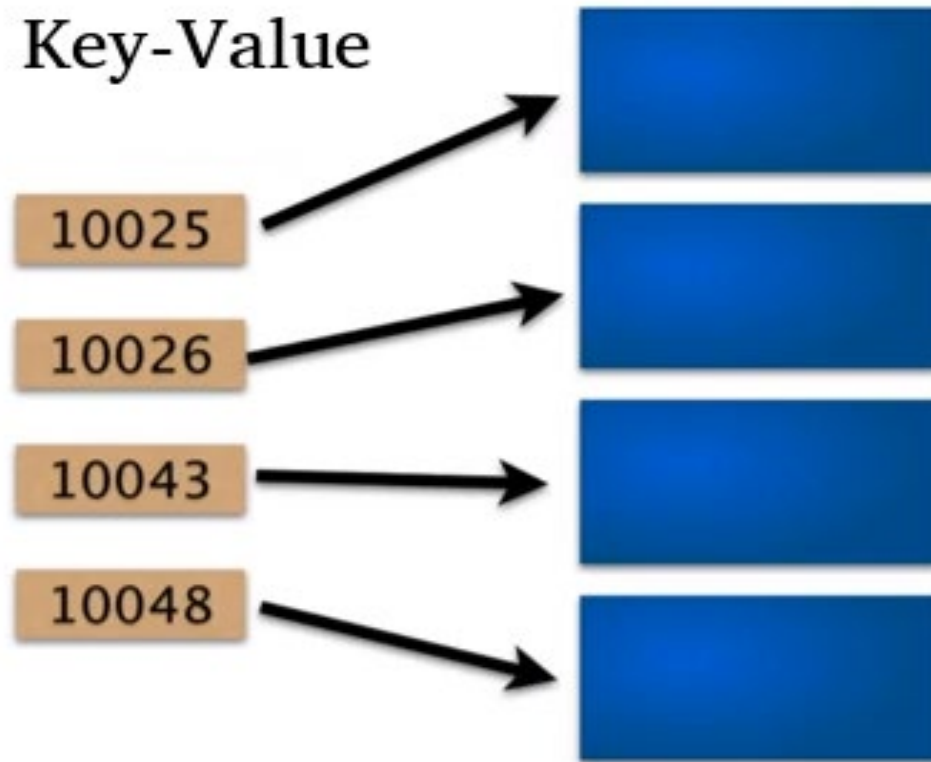
NoSQL database categories



Each category has:

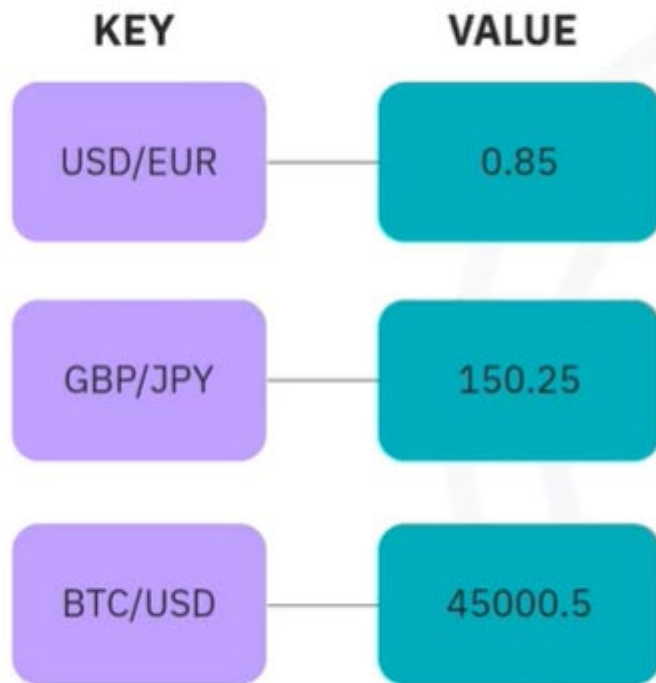
- Unique characteristics
- Architecture
- Use cases

Key-Value Data Model



Key-Value data model

Key-Value NoSQL database architecture



Key-Value NoSQL database architecture

KEY	VALUE
USD/EUR	0.85
GBP/JPY	150.25
BTC/USD	45000.5

- Least complex

Key-Value NoSQL database architecture

KEY	VALUE
USD/EUR	0.85
GBP/JPY	150.25
BTC/USD	45000.5

- Least complex
- Represented as a hashmap
- Ideal for basic CRUD operations

Key-Value NoSQL database architecture

KEY	VALUE
USD/EUR	0.85
GBP/JPY	150.25
BTC/USD	45000.5

- Least complex
- Represented as a hashmap
- Ideal for basic CRUD operations
- Scales well
- Shards easily

Key-Value NoSQL database architecture

KEY	VALUE
USD/EUR	0.85
GBP/JPY	150.25
BTC/USD	45000.5

- Not intended for complex queries

Key-Value NoSQL database architecture

KEY	VALUE
USD/EUR	0.85
GBP/JPY	150.25
BTC/USD	45000.5

- Not intended for complex queries
- Atomic for single key operations only
- Value blobs are opaque to database
 - Less flexible data indexing and querying

Data Stored in Redis:

Key	Value
<code>session:101</code>	<code>{ "user_id": 101, "name": "Sheersa", "role": "admin", "login": true }</code>
<code>session:102</code>	<code>{ "user_id": 102, "name": "Rahat", "role": "user", "login": true }</code>

These keys are strings like `session:101`, and values are stored as JSON strings.

Redis Commands

- # Set data
 - SET session:101 '{"user_id": 101, "name": "Sheersa", "role": "admin", "login": true}'
 - # Get data
 - GET session:101
 - # Update data (overwrite with new JSON)
 - SET session:101 '{"user_id": 101, "name": "Sheersa", "role": "admin", "login": false}'
 - # Delete session
 - DEL session:101
-

Key-Value NoSQL database use cases

Suitable use cases

- For quick basic CRUD operations on non-interconnected data

Key-Value NoSQL database use cases

Suitable use cases

- For quick basic CRUD operations on non-interconnected data
 - Storing and retrieving session information for web applications

Key-Value NoSQL database use cases

Suitable use cases

- For quick basic CRUD operations on non-interconnected data
 - Storing and retrieving session information for web applications
- Storing in-app user profiles and preferences

Key-Value NoSQL database use cases

Suitable use cases

- For quick basic CRUD operations on non-interconnected data
 - Storing and retrieving session information for web applications
- Storing in-app user profiles and preferences
- Shopping cart data for online stores

Key-Value NoSQL database use cases

Unsuitable use cases

- For data that is interconnected with many-to-many relationships
 - Social networks
 - Recommendation engines
- When high-level consistency is required for multi-operation transactions with multiple keys
 - Need a database that provides ACID transactions
- When apps run queries based on value vs. key

Key-Value NoSQL database examples

Popular key-value NoSQL databases include:

- Amazon DynamoDB

Key-Value NoSQL database examples

Popular key-value NoSQL databases include:

- Amazon DynamoDB
- Oracle NoSQL Database
- Redis

Key-Value NoSQL database examples

Popular key-value NoSQL databases include:

- Amazon DynamoDB
- Oracle NoSQL Database
- Redis
- Aerospike
- Riak KV, MemcacheDB
- Project Voldemort

Best Use Cases for Key-Value Stores

1. Session Management

- Store user session data during login/logout cycles.
- Ideal for fast access and temporary data.
- **Example:**
Store user login status, cart info, or preferences in Redis.

2. Caching

- Frequently accessed data (e.g., product info, API results) can be cached for ultra-fast reads.
 - **Example:**
Use Redis or Memcached to cache weather data, news headlines, or database query results.
-

Best Use Cases for Key-Value Stores

3. Shopping Cart Data

- Temporary, fast-changing, user-specific cart data can be stored using user ID as the key.
- **Example:**
cart:user123 → {items: ["book", "pen"], total: 12.99}

4. Real-time Analytics

- Track real-time metrics like views, likes, or counters.
 - **Example:**
Track post views: views:post123 → 4579
-

Best Use Cases for Key-Value Stores

5. User Preferences & Profiles

- Quickly fetch user profile info or settings using a unique user ID.

- **Example:**

user:987 → {theme: "dark", language: "en", notifications: true}

6. IoT Data Ingestion

- Devices send sensor data using unique device IDs. Fast write/read speed is essential.

- **Example:**

device:tempSensor001 → {"timestamp": 1712356, "value": 72.5}

Best Use Cases for Key-Value Stores

7. Feature Flag/Toggle Management

- Turn application features on/off dynamically using key-value pairs.
- **Example:**
feature:dark_mode → true

Key-Value Stores

Not Ideal For:

- Complex queries (e.g., "find all users from Canada")
 - Relational data with joins
 - Searching by values or filtering
-

Key-Value Data Model

- DynamoDB, Riak, Redis, Memcached DB, Berkeley DB and HamsterDB are some of the popular key-value data models.

Riak	RDBMS
Key	Row id
Bucket	Table
Key-Value	Row
Riak Cluster	Database instance

Comparisons of terminologies used in Oracle and Riak.

Column-Family data model

- In the Column-family based databases the data is stored in key-value format where each of the key is mapped to respective data values. These values are further assigned to multiple column families which act as a data map.
 - Some of the popular Column-family based databases are Cassandra, HBase, Hypertable, and Amazon SimpleDB.
-

Analogy: Student Report Cards (Row-wise) vs. Subject Notebooks (Column-wise)

- Row-wise Storage (Relational DBMS)
- Analogy: Imagine a stack of student report cards, and each report card contains all information about one student.
- Each sheet = one student
- You flip through sheets to find any specific data (like all GPAs)

Column-Family data model

- Example:
 - Sheet 1: Name: Alice | Age: 20 | GPA: 3.8
 - Sheet 2: Name: Bob | Age: 22 | GPA: 3.5
 - Sheet 3: Name: Carol | Age: 21 | GPA: 3.9
 - ✅ Easy to see everything about one student
 - ❌ Hard to analyze one field (like GPA) for all students — have to flip through all cards
-

Column-wise Storage (NoSQL Column DB)

- Analogy: Imagine 3 notebooks, one for Name, one for Age, and one for GPA. Each notebook has data for all students for that one attribute.
- Notebook 1 (Name): Alice, Bob, Carol
- Notebook 2 (Age): 20, 22, 21
- Notebook 3 (GPA): 3.8, 3.5, 3.9

Column-wise Storage (NoSQL Column DB)

- If you're only interested in GPAs, you just open the GPA notebook and read down the list, super fast and efficient.
 - ✅ Great for analyzing one column across many students
 - ❌ Takes more effort to reconstruct a full student profile
-

Row Store (RDBMS):

Each student's full record is stored together:

```
pgsql
```

Name	Age	GPA	
-----	-----	-----	
Alice	20	3.8	
Bob	22	3.5	
Charlie	19	3.9	

Column Store (NoSQL):

All **Names** are stored together, all **Ages** together, and all **GPA**s together:

```
pgsql
```

```
Column: Name    -> Alice, Bob, Charlie
Column: Age      -> 20, 22, 19
Column: GPA      -> 3.8, 3.5, 3.9
```

Ride Records in a Ride-Sharing App

Each ride in a ride-sharing app includes information like:

- Rider Name
 - Driver Name
 - Pickup Location
 - Dropoff Location
 - Fare
 - Date & Time
-

Row-wise Storage (Relational DBMS)

Analogy:

Imagine the ride-sharing company keeps a **logbook**, where **each row** is a **complete ride record**:

Rider	Driver	Pickup	Dropoff	Fare	Date & Time
Alice	Bob	A	B	\$15	10:00 AM
John	Sara	X	Y	\$22	10:10 AM
Emma	Max	M	N	\$30	10:15 AM

-  Good if you want to see **all details of a single ride**
-  Slow if you want to get **only fares** for the last million rides

Column-wise Storage (NoSQL Column DB)

Analogy:

Instead of a logbook, imagine **6 notebooks**, one for each column of ride data:

- One notebook only for Rider Names: Alice, John, Emma
- One for Driver Names: Bob, Sara, Max
- One for Pickup Locations: A, X, M
- One for Dropoff Locations: B, Y, N
- One for Fares: \$15, \$22, \$30
- One for Date & Time: 10:00 AM, 10:10 AM, 10:15 AM

Now, if the **analytics team** wants to:

“Calculate the average fare of all rides taken today.”

- ✓ You can directly access the **Fare notebook** — no need to touch other data
- ✓ Much faster for analyzing one type of information (e.g., fare trends, peak hour patterns)

Comparison Table:

Use Case	Row-wise (RDBMS)	Column-wise (NoSQL Column DB)
See complete details of a specific ride	Efficient – full row is together	Slower – needs to gather from multiple cols
Analyze total fare collected today	Slower – must read entire rows	Fast – read only the "Fare" column
Most useful for...	Transactional systems (booking)	Analytical systems (fare insights, trends)

Step 1: Define the Table Schema

Here's how you might define a wide-column table in Cassandra to store ride data:

```
CREATE TABLE rides_by_date (  
  ride_date date,  
  ride_id uuid,  
  driver text,  
  rider text,  
  pickup_location text,  
  dropoff_location text,  
  distance float,  
  fare decimal,  
  PRIMARY KEY (ride_date, ride_id)  
);
```

Step 2: Insert Data

```
INSERT INTO rides_by_date (  
    ride_date, ride_id, driver, rider,  
    pickup_location, dropoff_location, distance, fare  
) VALUES (  
    '2025-04-11', uuid(), 'Bob', 'Priya',  
    'Seattle', 'Redmond', 12.0, 18.00  
);
```

Step 3: Query Example – Total Fare on a Date

You'd typically fetch data and sum it in application logic:

```
SELECT fare FROM rides_by_date WHERE ride_date = '2025-04-11';
```

Then in your application (e.g., Java, Python), you'd sum the returned fares.

If using Spark with Cassandra, you can even write:

```
rides_df.filter("ride_date = '2025-04-11']").agg(sum("fare")).show()
```

Column-Family data model

- Cassandra is the commonly used column-family based database.

RDBMS	Cassandra
Database	Keyspace
Row	Row
Table	Column Family
Database instance	Cluster
Column (same throughout the rows)	Columns (can be unique for each row)

Comparisons of terminologies used in RDBMS and Cassandra

Document data model

- Documents are at the core of document databases. To understand the working of document model let us consider, a bill generated by an e-commerce store.
 - **A bill contains following details:** order number, customer number, the detail of items purchased and the payments summary. In case of the relational model, to generate this bill we have to join multiple tables.
 - For example, the order number will be extracted from order table, customer detail may be extracted from customer table, the detail of items purchased from order detail table and payment details from credit card table.
-

Document Store as a Digital Folder of Resumes

Imagine you're an HR manager...

You receive **resumes from job applicants**, and each resume is a **self-contained document** like a PDF or Word file with different sections such as:

- Name
 - Contact Info
 - Work Experience
 - Education
 - Skills
 - Each applicant formats their resume slightly differently some have 2 degrees, others have 5 skills, some add certifications, others don't.
-

Document Store

Each **resume** = **one document**, typically stored in **JSON** or **BSON** format.

Each **applicant** = **one record (document)**, but there's **no fixed schema**.

Example document:

```
json

{
  "name": "Alice",
  "email": "alice@example.com",
  "experience": [
    { "company": "ABC Corp", "role": "Developer" }
  ],
  "skills": ["Python", "MongoDB", "React"]
}
```

Document Store

Another document:

```
json

{
  "name": "Bob",
  "email": "bob@example.com",
  "certifications": ["AWS Certified"],
  "experience": [],
  "portfolio": "bob.dev"
}
```

Document Store:: Key Takeaways

Concept	Relational DB	Document Store (NoSQL)
Structure	Fixed schema (tables)	Flexible schema (documents)
Real-life example	Database of Excel sheets	Folder of unique resumes
Uniformity	Every row has same fields	Each document can vary
Best for	Structured, consistent data	Semi-structured, nested data

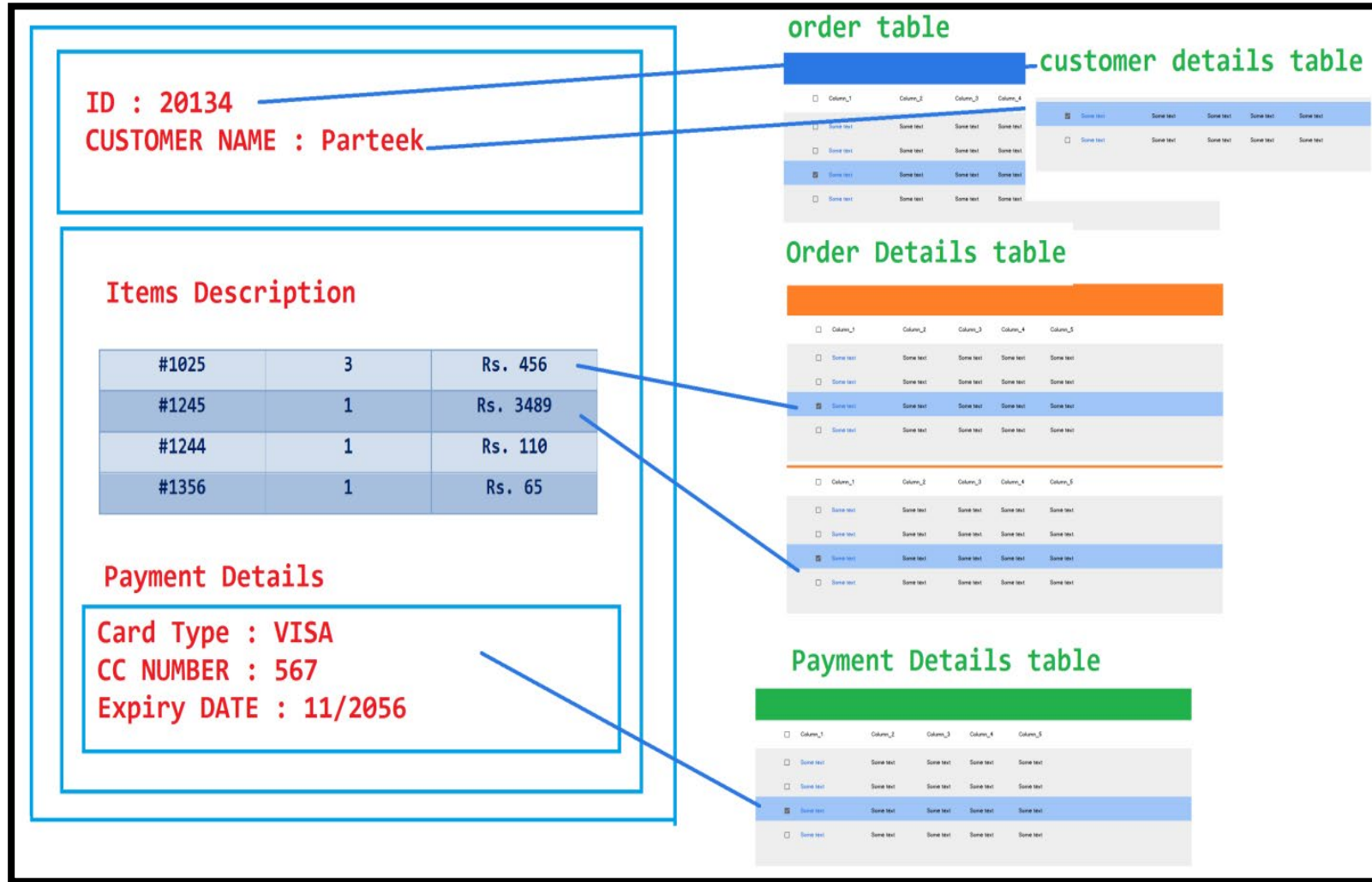
Use Cases for Document Store:

- E-commerce product catalog (every product has different fields)
 - User profiles in a social media app
 - Blog posts with comments, tags, and metadata
 - IoT device data with flexible sensor readings
-

Document data model

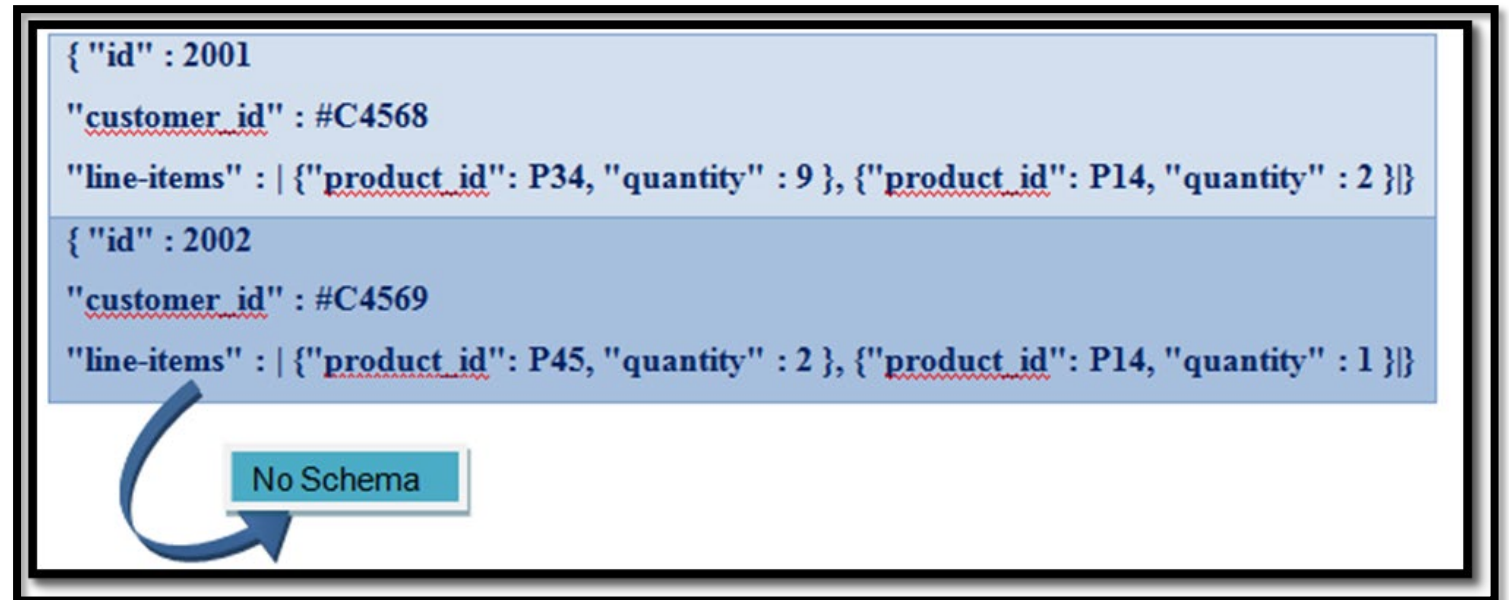
- In case of a cluster environment, these tables may exist on different computers and running a join query across multiple computers is not very efficient.
 - Alternatively, we can store this bill document itself in the database on a single computer. Of course, it would not be normalized but is a more efficient approach in case of the cluster environment.
 - This type of storage comes under document data model. It is non-relational and usually supports in the JSON, XML, CSV, OR TSV formats.
-

Document data model



Document data model

- In document data model, the database stores and retrieves documents which are self describing, use hierarchical tree data structures and does contain collections, maps and scalar values.



Document data model

Document Store

Product 1 – Book:

```
{  
  "_id": "book123",  
  "name": "The Great Gatsby",  
  "category": "book",  
  "author": "F. Scott Fitzgerald",  
  "pages": 180,  
  "price": 10.99  
}
```

Product 2 – Smartphone:

```
{  
  "_id": "phone456",  
  "name": "Pixel 8",  
  "category": "electronics",  
  "brand": "Google",  
  "specs": {  
    "ram": "8GB",  
    "storage": "128GB"  
  },  
  "price": 699.00  
}
```

Document Store

Querying in MongoDB (Mongo Shell or Compass):

Find all books:

```
db.products.find({ category: "book" })
```

Find products priced under \$100:

```
db.products.find({ price: { $lt: 100 } })
```

Find phones with 128GB storage:

```
db.products.find({ "specs.storage": "128GB" })
```

Why Use Document Store Here?

- Product structure varies across categories.
 - Adding new fields (e.g., battery_life for electronics) doesn't need schema changes.
 - Very agile for development perfect for startups and evolving apps.
-

Document data model

- MongoDB is the commonly used document data model based software.
- Comparisons of terminologies used in MongoDB and RDBMS.

MongoDB	RDBMS
Database	schema
Collection	Table
_id	Row id
DBRef	Joins
MongoDB instance	Database instance
document	Row

Graph Databases

- Graph databases have existed long before the term "NoSQL" was coined.
 - They have a special place for the graph Databases.
 - NoSQL databases were developed to perform computation on clusters, as a result of which aggregate-oriented data models were introduced which consisted of large records with simple connections.
-

Graph Databases

Graph Database as a Social Network

Imagine you're on a platform like Facebook or LinkedIn

You have:

- **People (nodes)**
 - **Relationships (edges)** like “friend,” “follows,” “works at,” “lives in”
 - And each person and relationship can have **properties**
-

Graph Databases

Let's say you want to model:

- Alice is friends with Bob
- Bob works at Google
- Alice lives in Seattle
- Bob lives in New York

This would look like:

SCSS

```
(Alice) -[FRIENDS_WITH]→ (Bob)
(Bob) -[WORKS_AT]→ (Google)
(Alice) -[LIVES_IN]→ (Seattle)
(Bob) -[LIVES_IN]→ (New York)
```

Each circle = **Node**

Each arrow = **Edge/Relationship**

Graph Databases

Why is this Powerful?

- Suppose you ask:
“Who are Alice’s friends that live in New York and work at Google?”
- With a relational DB, you’d need multiple joins across tables: users, friendships, companies, locations.
- With a graph DB, you simply traverse the graph follow the arrows. It’s natural, fast, and efficient.

Key Takeaways

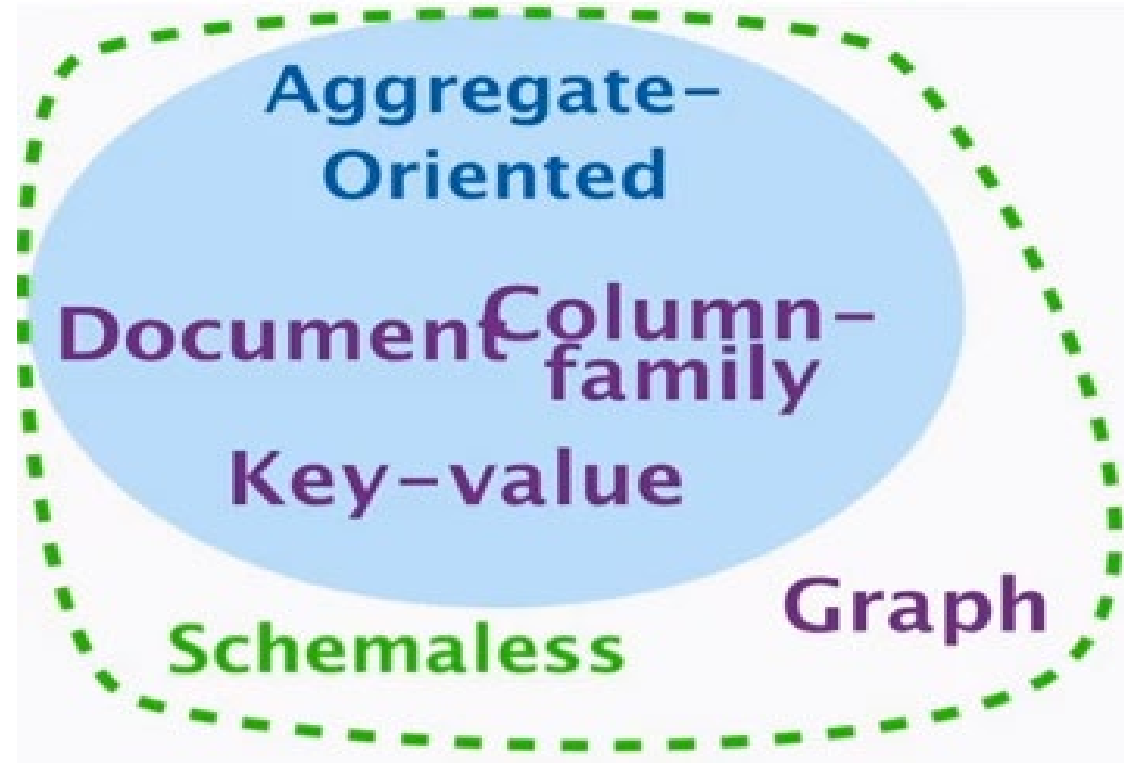
Feature	Graph Database	Analogy
Data Structure	Nodes + Relationships	People + Friendships in a social network
Querying	Traversal	Following connections
Strength	Relationship-heavy data	Social networks, fraud detection, maps
Schema	Flexible	People can have different attributes

Real-Life Examples of Graph DB Usage:

- Finding shortest path in maps (Google Maps, Uber)
 - Recommending friends or products (LinkedIn, Amazon)
 - Detecting fraud rings in financial networks
 - Analyzing supply chain dependencies
-

Data Models of NoSQL

- Key-value data model
- Document data model
- Column-family model
- Graph data model



Graph Databases

- Let us consider the case of social networking platform such as Facebook that maintains a social graph for each user.
- In case of the relational model, we can store friend's information in FRIEND table that will contain the user id of a friend.

UserID	Friend_UserID
100	102
102	101
101	102
100	103
103	104
104	102
102	105

Graph Databases

- in order to perform any activity such as fetching updates from friend's wall, we will need to write a complex query involving multiple joins.
 - In such a scenario, it is better to store graph itself in the database. In graph data database we have nodes that are connected by edges.
 - Once the graph containing nodes and edges is built, the database will allow you to query this network and apply operations that work with a graph data structure.
-

Graph Databases

- This makes graph different from the relational databases. Relational databases do come up with the functionality of Foreign Keys, but the joins required for navigating are a costly business and often results into poor performance.
- Whereas, the graph databases make traversal along the relationships very light-weight.
- Neo4J is a popular database based on graph data model. Creating a graph doesn't cost much effort in fact it is as simple as creating nodes followed by declaring a relationship among nodes.

Graph Databases

- For instance, let us create two nodes, Rahat and Rishan:

- ❑ `Node rahat = graphDb.createNode();`
- ❑ `rahat.setProperty("name", "Rahat");`
- ❑ `Node rishan = graphDb.createNode();`
- ❑ `rishan.setProperty("name", "Rishan");`

- Creating Relationship

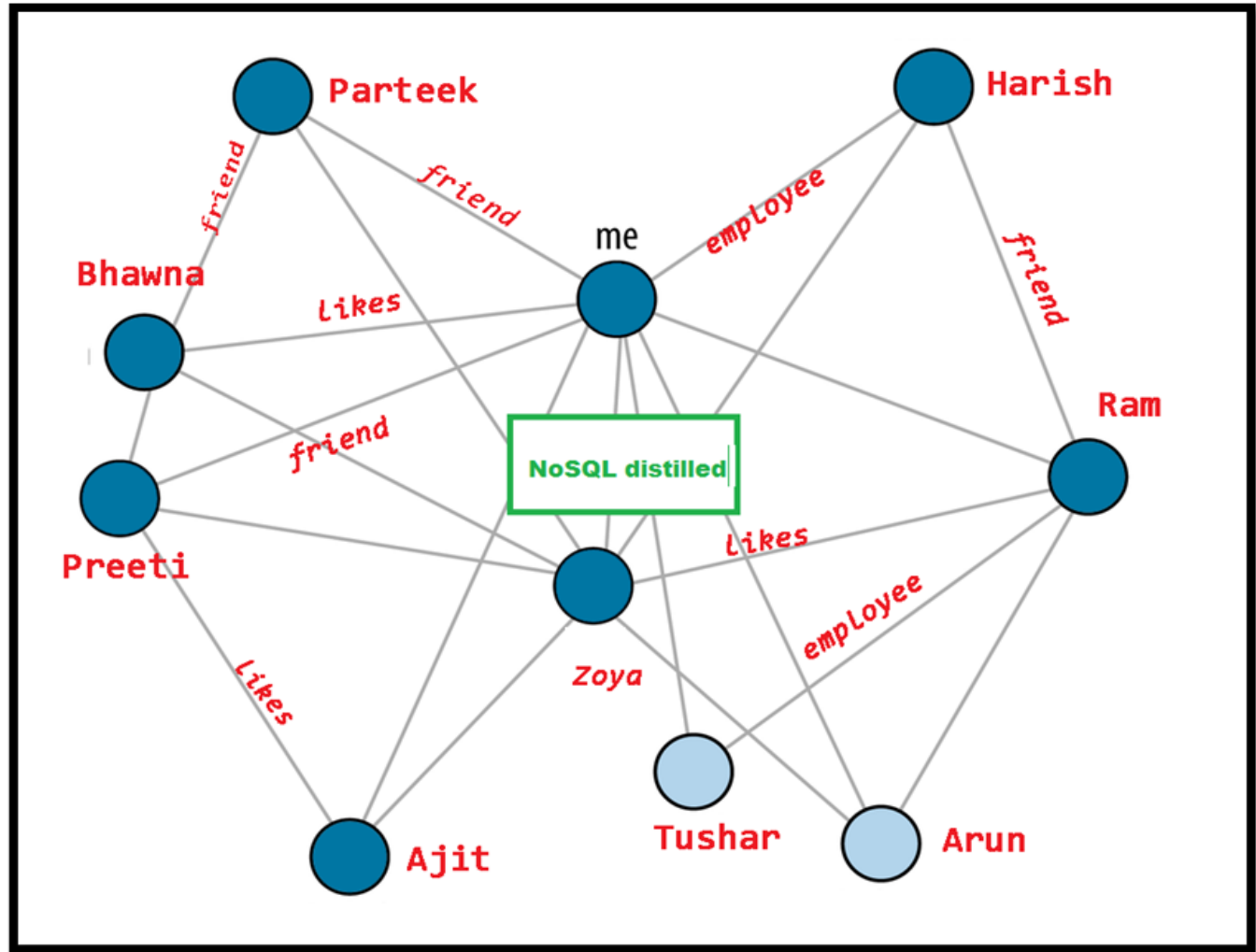
- ❑ `rahat.createRelationshipTo(rishan, FRIEND);`
- ❑ `rishan.createRelationshipTo(rahat, FRIEND);`

Graph Databases

- we have an information web where the number of nodes are very less, but there is a rich structure of interconnections between them.
 - This makes it a perfect choice for capturing data consisting of complex relationships as in the case of data found through social networks, product preferences, or eligibility rules.
 - Thus, Graph databases store entities and relationships between these entities.
 - Entities are nothing but nodes which can have different properties.
 - Relations resemble edges that may have properties. Edges of graph may have some directional significance attached to them.
-

Graph Databases

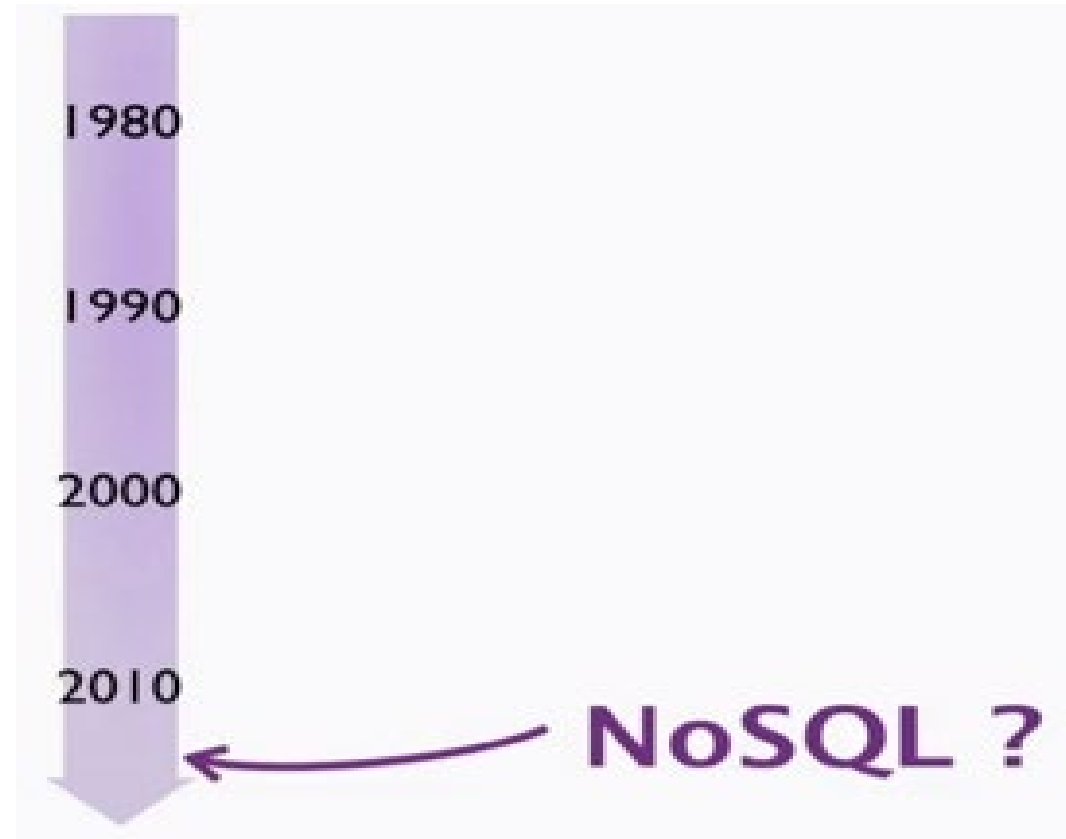
- An example of graph structure.



Future of NoSQL

- After discussing features and models of NoSQL, we can conclude that NoSQL makes software development process easier and can handle large-scale data.
 - But a billion-dollar question is - "What will be the future of NoSQL."
-

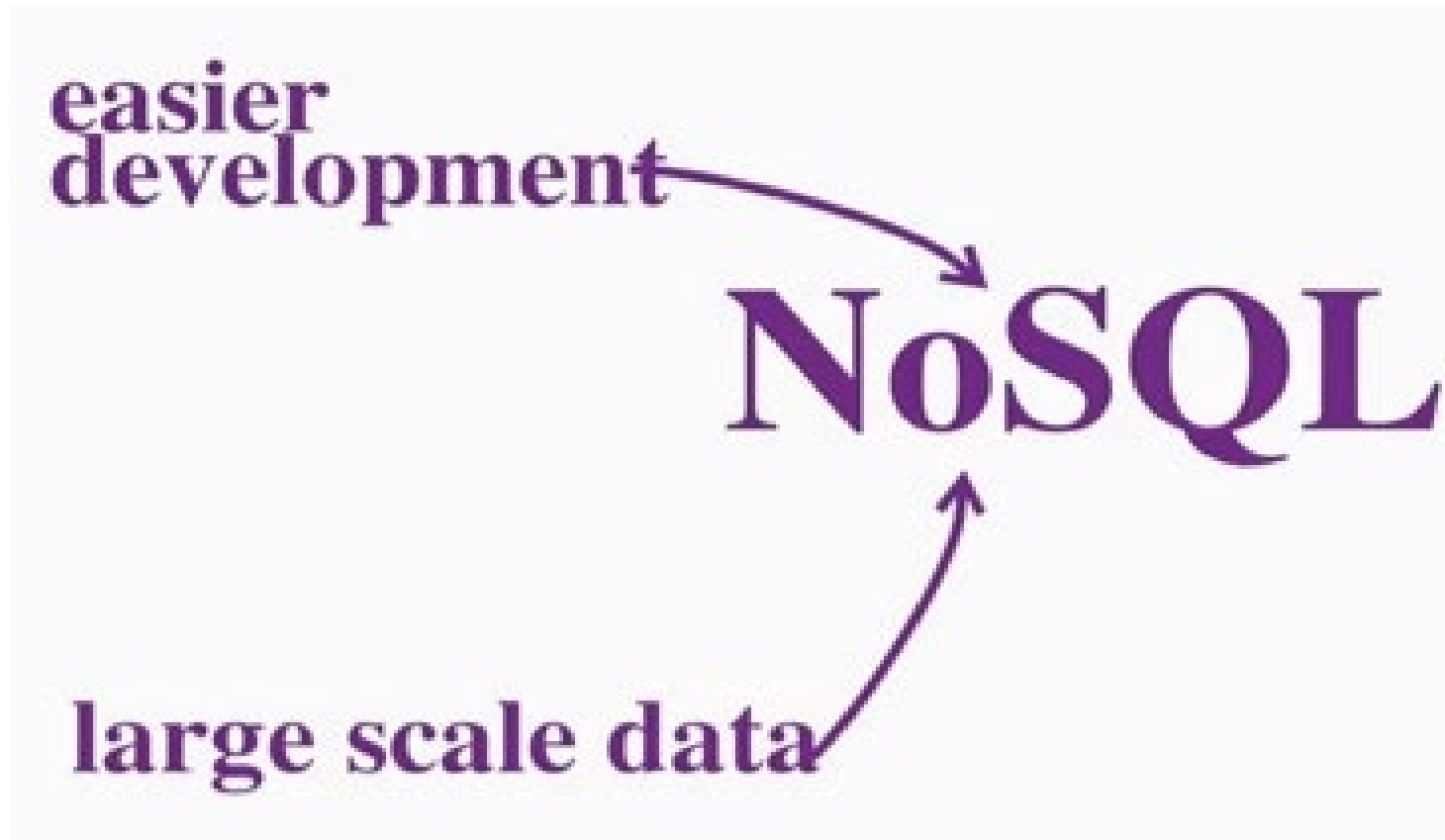
Future of NoSQL



Future of NoSQL

- The major strength of the relational model is ACID properties.
 - It means that data is always consistent, but NoSQL works in the shadow of CAP theorem.
 - In a financial application, we cannot compromise on the consistency of data, so in future, we will continue to use relational model for all financial applications.
 - But, in applications like an e-commerce where availability is the major concern NoSQL data models perform better than relational model because of ease of development and ability to handle large-scale data.
-

Future of NoSQL

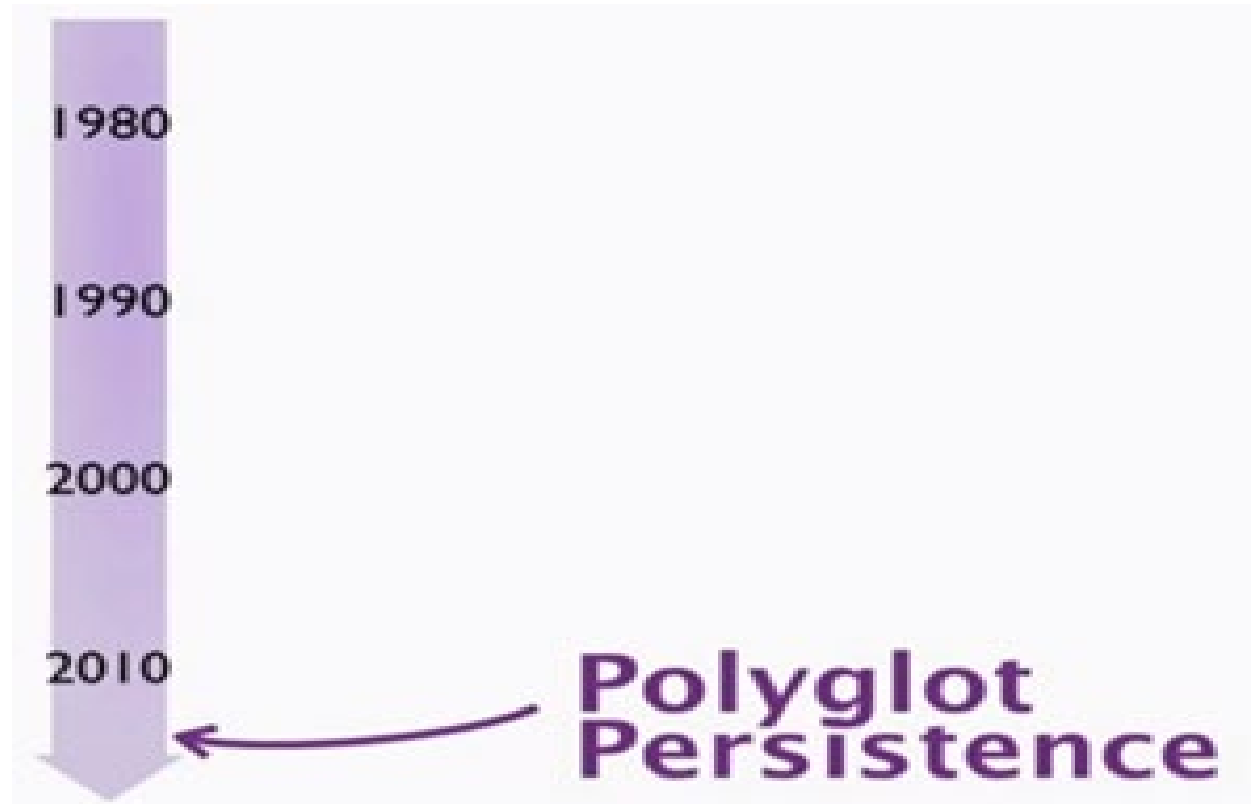


Strength of No-SQL

Future of NoSQL

- Similarly, in applications like social networking websites where queries are based on users and their activities, graph-based models performed better. So, in future, we will mix different technologies i.e. the programmers will use the right technology for doing some specific job in hand.
 - Thus in the field of Big Data a new term has been emerged, which is called “polyglot persistence” . In order to define this hybrid approach we use polyglot persistence.
-

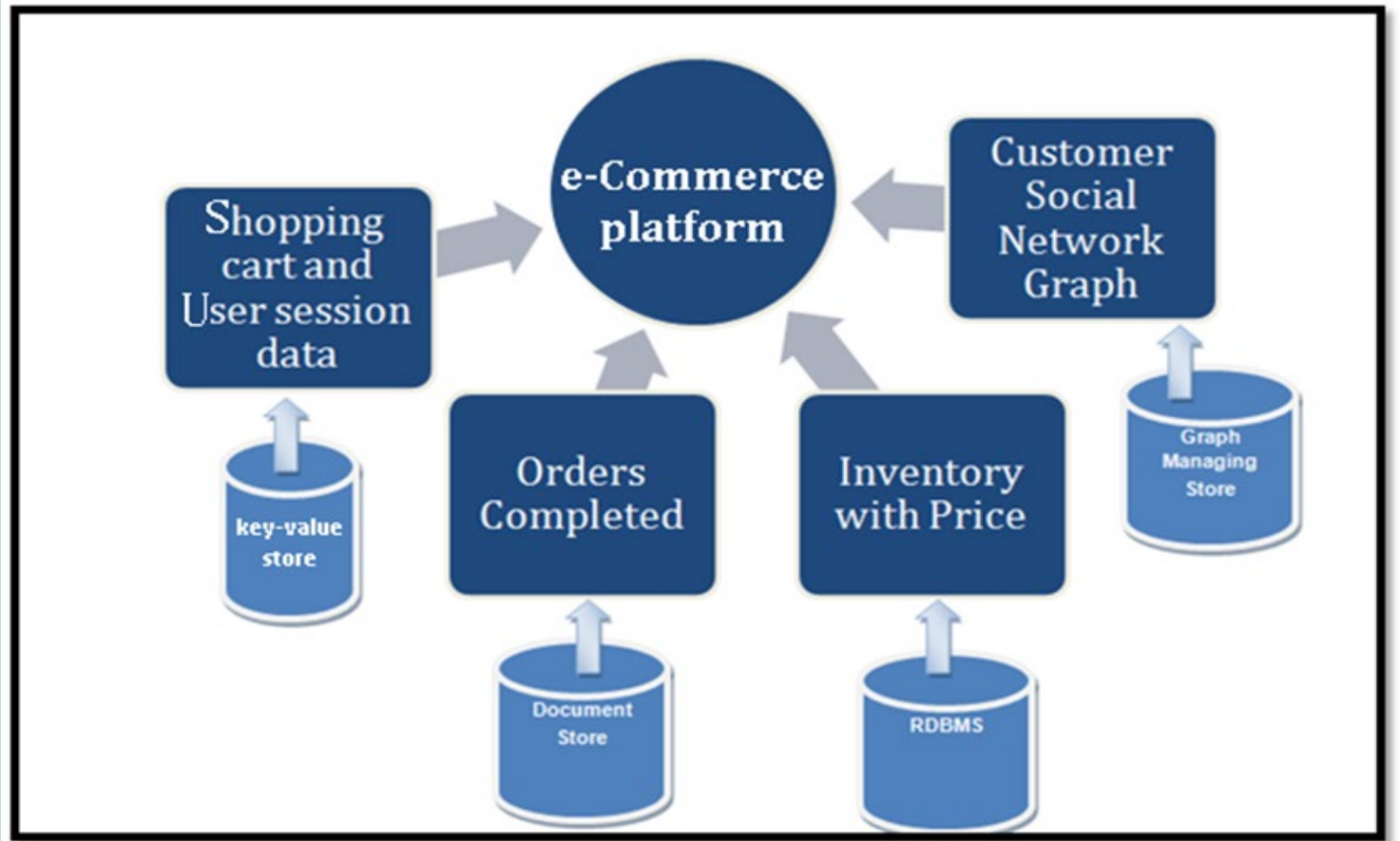
Future of NoSQL



Future is Polyglot Persistence.

Future of NoSQL

- For instance, an e-commerce platform may require a hybrid approach to information.
- Polyglot Persistence in a real environment.



Future of NoSQL

- For shopping cart and session data, the key-value store is the best-suited model; to store completed order information document model is best while for inventory and item price relational model is optimal.
- For customer's social graph, a graph-based data model is best suited. Thus, in future, we will see lots of hybrid data models being used.
- In nutshell, we can conclude that in future relational model and NoSQL both co-exist and will be used in a hybrid manner to provide full e-commerce solutions.

Distinguishing between NoSQL and Relational Data Models (RDBMS)

- There are two major differences between NoSQL and RDBMS.
 - First, RDBMS uses a relational data model whereas NoSQL provides us with a bunch of options, each suited for a different scenario. So in an RDBMS, we put the data in a bunch of tables and query it using complex join operations to present useful information to the user.
 - On the other hand, in NoSQL, the data may be stored in aggregate form keeping software development as simple as possible.
-

Distinguishing between NoSQL and Relational Data Models (RDBMS)

- Secondly, RDBMS requires a rigid schema where NoSQL is schema-less. Once the values are entered in a relational database it is a very tough deal to change the database design, in fact it should be frequently avoided – the exact contra of the behavior is the basic requirement in the Big Data era, where application developers bring out several changes in database design and incorporate different data types.
- In comparison, NoSQL allows changing the schema or the format of the data being inserted any time, without application disruption.
- Though even NoSQL doesn't guarantee the full ACID (atomicity, consistency, isolation, durability) property but it still has an architecture for fault tolerance and distributed computing.

Distinguishing between NoSQL and Relational Data Models (RDBMS)

- Though even NoSQL doesn't guarantee the full ACID (atomicity, consistency, isolation, durability) property but it still has an architecture for fault tolerance and distributed computing.